

Proyecto Temporis

Desarrollo de
Aplicaciones
Multiplataforma



Alumno:
Kevin Zamora Amela

Fecha:
10/06/2026

Índice

0. Resumen ejecutivo.....	4
Descripción del proyecto.....	4
Qué se espera lograr con la implementación para el usuario final.....	4
Por qué es importante desarrollar esta aplicación.....	4
1. Identificación del producto.....	5
1.1. Análisis del sector y estructura de la empresa.....	5
1.2. Estudio de mercado.....	7
1.3. Innovación, dificultad e investigación.....	8
1.4. Ayudas y subvenciones.....	9
1.5. Especificación de requisitos.....	10
Requisitos Funcionales (RF).....	10
Requisitos No Funcionales (RNF).....	11
1.6. Casos de uso.....	12
Identificación de Actores.....	12
Especificación Detallada de Casos de Uso Críticos.....	12
1.7. Bocetos de interfaz.....	15
Estructura y Disposición de las Vistas.....	15
Decisiones de Diseño Justificadas para la Accesibilidad Real.....	15
2. Diseño de la aplicación.....	17
2.1. Estudio de viabilidad.....	17
2.2. Planificación, ciclo de vida y diagrama Gantt.....	19
Resumen del Alcance del Mínimo Producto Viable (MVP) y sus Exclusiones....	20
Representación Temporal del Proyecto (Diagrama de <i>Gantt</i>).....	21
2.3. Entidad Relación.....	22

Descripción de las Entidades y Naturaleza <i>NoSQL</i>	23
2.4. Esquema Relacional.....	24
2.5. Diagrama de Clases.....	27
2.6. Diseño de Interfaces.....	29
2.6.1. Sistema de Diseño Visual (Visual Design System).....	29
2.6.2. Componentes Reutilizables de la Interfaz.....	29
2.6.3. Catálogo de Vistas de la Aplicación.....	30
2.7. Plan de Pruebas.....	37
2.7.1. Definición del Alcance y Bloques Críticos de Prueba.....	37
2.7.2. Matriz de Casos de Prueba Escenciales.....	37
2.7.3. Reporte de Cobertura de Código (<i>Code Coverage</i>) y Métricas de Calidad	38
3. Gestión de proyecto - Planificación.....	40
3.1. Planificación Proyecto - Scrum.....	40
3.1.1. <i>Product Backlog</i> (Pila del Producto).....	40
3.1.2. Desarrollo de los <i>Sprints</i> de Ejecución.....	40
3.1.3. Tablero Kanban (<i>GitHub Projects</i>).....	41
4. Gestión del proyecto - Seguimiento.....	42
4.1. Seguimiento Proyecto - Scrum.....	42
4.1.1. Registro y Trazabilidad de Incidencias Técnicas.....	42
4.1.2. Procedimiento de Gestión de Cambios.....	43
4.1.3. Retrospectiva Final del Proyecto.....	43
5. Bibliografía y Webgrafía.....	44
5.1. Documentación Oficial y Frameworks de Desarrollo.....	44
5.2. Estándares de Diseño y Accesibilidad Universal.....	44
5.3. Calidad de Software, Testing y Herramientas de Entorno.....	44

0. Resumen ejecutivo

Descripción del proyecto

El proyecto **Temporis** consiste en el diseño, desarrollo e implementación de una aplicación móvil nativa para el sistema operativo Android (programada en *Kotlin* y *Java*). Su propósito principal es ofrecer una solución integral para la gestión del tiempo operativo, el control de intervalos de concentración y la optimización de la productividad personal y laboral mediante un sistema flexible de temporizadores y contadores personalizados. A nivel de infraestructura de datos, la aplicación delega la persistencia de información y el control perimetral en la suite en la nube de *Google Firebase* (*Authentication* y *Cloud Firestore*).

Qué se espera lograr con la implementación para el usuario final

- **Gestión eficiente del tiempo:** Permitir a los usuarios monitorizar con precisión matemática los bloques de tiempo dedicados a tareas u objetivos específicos mediante mecánicas de cuenta regresiva personalizables.
- **Accesibilidad Universal (Inclusión real):** Proporcionar una interfaz reactiva y dinámica que cumpla con las pautas *WCAG* (*Web Content Accessibility Guidelines*), permitiendo la adaptación de la interfaz en tiempo real (escalado de fuentes, alto contraste) para usuarios con diversidad funcional o neurodivergencia.
- **Persistencia y Sincronización:** Asegurar que los datos del historial de tiempos estén siempre disponibles en tiempo real en cualquier dispositivo, garantizando la resiliencia en entornos con conectividad variable gracias al soporte *offline* del almacenamiento local indexado.

Por qué es importante desarrollar esta aplicación

El mercado actual del *software* de productividad cuenta con herramientas robustas, pero adolece de una ineficiencia operativa crítica: la exclusión sistemática de usuarios con capacidades diversas debido a interfaces rígidas y complejas. Como proyecto impulsado por la vivencia de la neurodivergencia (TDA y Dislexia) y el Daño Cerebral Adquirido (DCA), **Temporis** nace para demostrar que el software de alto rendimiento y la accesibilidad nativa no son mutuamente excluyentes. Su desarrollo es vital para dotar a los entornos corporativos inclusivos y al público general de una herramienta que elimine barreras de usabilidad, transformando el control del tiempo en una experiencia cómoda, intuitiva y justa para todos.

1. Identificación del producto.

1.1. Análisis del sector y estructura de la empresa

El proyecto se enmarca dentro del sector de las Tecnologías de la Información y la Comunicación (TIC), específicamente en el subsector estratégico del **Desarrollo de Software para la Productividad y la Gestión de Tiempos**. En este ecosistema, la gestión precisa del tiempo operativo se ha consolidado como la métrica clave para la rentabilidad de las organizaciones modernas.

Para definir el espectro donde operará e impactará **Temporis**, se categorizan tres modelos organizacionales diana según sus necesidades específicas, estructuras departamentales e interacción funcional con la herramienta:

A. Startup de Desarrollo Tecnológico "CodeFocus" (Sector IT - Mediana Empresa)

- **Modelo de Negocio:** Desarrollo de soluciones de software a medida bajo demanda.
- **Necesidad Operativa:** Control estricto del cómputo de horas por proyecto para garantizar una facturación exacta y auditable al cliente final.

Estructura Funcional e Integración:

- *Dirección (CEO):* Define los objetivos de rentabilidad anual del negocio.
- *Gestión de Proyectos (PM):* Supervisa plazos e hitos; utiliza las métricas de **Temporis** para contrastar el tiempo estimado frente al tiempo de desarrollo real.
- *Desarrollo (IT):* Programadores que ejecutan el código; emplean la aplicación nativa para mantener el enfoque (*Ej. Técnica Pomodoro: Método de gestión del tiempo que mejora la concentración y evita la fatiga mental. Consiste en dividir el trabajo en intervalos ininterrumpidos de 25 minutos [llamados pomodoros] seguidos de descansos breves, con pausas más largas tras cuatro ciclos completados*) y persistir sus bloques de trabajo en la nube.
- *Administración:* Extrae de manera automatizada los registros temporales consolidados para emitir facturas precisas.

B. Academia de Formación "E-Learn Innovate" (Sector Educativo - Microempresa/Autónomos)

- **Modelo de Negocio:** Plataforma de capacitación virtual y tutorías personalizadas.
- **Necesidad Operativa:** Fomentar hábitos de estudio eficientes, estructurados y saludables en un alumnado heterogéneo.
- **Estructura Funcional e Integración:**
 - *Coordinación Académica:* Diseña los planes de estudio y prescribe el uso de **Temporis** para la segmentación de contenidos en bloques de enfoque de 45 minutos.

Proyecto Temporis - Desarrollo de Aplicaciones Multiplataforma

- *Departamento de Tutorías:* Utiliza la configuración de accesibilidad dinámica de la aplicación para guiar a estudiantes con barreras visuales o cognitivas.
- *Soporte Técnico:* Integra las guías y flujos de la aplicación en el manual de bienvenida del alumno.

C. Centro de Inclusión Laboral "*IntegraTech*" (Sector Social - Gran Empresa / Centro Especial de Empleo)

- **Modelo de Negocio:** Servicios de externalización (*outsourcing*) de procesos de datos gestionados por personal con diversidad funcional.
- **Necesidad Operativa:** Herramientas de producción adaptadas al 100% que garanticen la igualdad de condiciones en el puesto de trabajo.
- **Estructura Funcional e Integración:**
 - *Unidad de Apoyo a la Inclusión:* Especialistas y psicólogos que adaptan los terminales; configuran el Alto Contraste y el Tamaño de Fuente dentro de **Temporis** según la necesidad del operario.
 - *Operaciones (Núcleo de Servicios):* Los trabajadores monitorizan el rendimiento y autogestionan sus descansos obligatorios.
 - *Recursos Humanos:* Valora la seguridad del *Login Biométrico* para proteger datos sensibles de clientes y facilitar el acceso a usuarios con dificultades motoras.

1.2. Estudio de mercado

El estudio de mercado se ha estructurado mediante la técnica de *Benchmarking* y el diagnóstico estratégico para identificar el hueco de mercado óptimo del producto.

Evaluación de la Competencia (*Benchmarking*)

Solución Competidora	Fortalezas Estratégicas	Debilidades Críticas
Toggl Track	Ecosistema multiplataforma muy maduro; herramientas corporativas completas.	Interfaz excesivamente densa y compleja; accesibilidad nativa nula o muy limitada.
Forest	Excelente nivel de gamificación; gancho psicológico de retención.	Enfoque marcadamente lúdico; ineficaz para entornos corporativos o industriales formales.
Clockify	Modelo gratuito y flexible para la gestión de equipos de trabajo.	Privacidad de datos cuestionable; dependencia absoluta de conexión a internet continua.
TEMPORIS	Accesibilidad nativa profunda, seguridad biométrica integrada y simplicidad conceptual extrema.	Producto de nueva marca en fase de despliegue inicial de Mínimo Producto Viable (MVP).

Matriz de Diagnóstico DAFO

- **Debilidades (D):** Marca nueva sin experiencia ni presencia previa en tiendas de aplicaciones; equipo de desarrollo unificado en un único desarrollador.
- **Amenazas (A):** Presencia de competidores consolidados con presupuestos masivos de marketing; alta velocidad de actualización de las APIs de accesibilidad base en Android.
- **Fortalezas (F):** Arquitectura orientada al 100% a la inclusión (normativas WCAG); integración nativa reactiva con la suite de Google Firebase; bajo consumo de memoria y recursos en el dispositivo móvil.
- **Oportunidades (O):** Adopción masiva del teletrabajo y la autogestión de tareas; endurecimiento de las normativas legales internacionales de inclusión laboral; auge en el consumo de aplicaciones de bienestar y salud mental.

1.3. Innovación, dificultad e investigación

El valor técnico e innovador de **Temporis** se sustenta sobre la resolución de problemas complejos de ingeniería de *software* para plataformas móviles.

Innovación y Propuesta de Valor Único

A diferencia de los enfoques tradicionales que delegan la accesibilidad en las capas de configuración global del sistema operativo, **Temporis** integra las pautas de accesibilidad en el núcleo reactivo de la aplicación. Destaca su **Diseño Reactivo de Interfaz**, capaz de reajustar los coeficientes de contraste térmico y escalas tipográficas en tiempo real mediante *sliders* internos, así como la capacidad de detectar dinámicamente el despliegue del teclado virtual para asegurar que ningún campo de control de tiempos quede oculto o inaccesible al usuario.

Dificultad Técnica y Retos Superados

1. **Sincronización *Cloud* Concurrente:** Control de estados reactivos e hilos asíncronos para evitar condiciones de carrera al pausar o inicializar contadores, garantizando la consistencia inmediata en la base de datos distribuida *NoSQL*.
2. **Abstracción de Capa Biométrica:** Implementación avanzada de la *API BiometricPrompt* de Android para gestionar de forma segura múltiples niveles de *hardware* criptográfico (reconocimiento facial, sensor dactilar) a través de distintas *API-levels* del sistema operativo.
3. **Procesamiento Gráfico Avanzado:** Manipulación del motor de renderizado de la librería de terceros *MPAndroidChart* para construir una matriz de contribución anual (rejilla indexada de 12 meses por 7 días) mediante estructuras de datos de tipo *BarEntry* apiladas y formateadores personalizados (*ValueFormatter*) para la internacionalización de datos.

Líneas de Investigación Aplicadas

- **Arquitectura limpia MVVM:** Migración de la lógica imperativa basada en *findViewById* hacia un desacoplamiento estricto empleando la arquitectura *Model-View-ViewModel* con *ViewBinding*, asegurando la escalabilidad del mantenimiento del código.
- **Seguridad Perimetral NoSQL:** Diseño analítico de colecciones documentales y reglas de escritura/lectura perimetrales (*Firebase Security Rules*) basadas en *tokens* criptográficos (`request.auth.uid`).

1.4. Ayudas y subvenciones

La viabilidad financiera y el despliegue a gran escala de **Temporis** en el mercado laboral se asienta sobre tres programas de financiación institucional plenamente alineados con la naturaleza del software:

1. Programa Kit Digital (Fondos Next Generation EU - Red.es):

- *Aplicación:* Orientado a microempresas y autónomos que deseen adoptar la aplicación para la digitalización de sus métricas de rendimiento interno bajo la categoría de "Gestión de Procesos" o "*Business Intelligence*". Financia la implantación de la solución móvil a través de los bonos digitales estipulados.

2. Ayudas NEOTEC (Centro para el Desarrollo Tecnológico Industrial - CDTI):

- *Aplicación:* Dirigido a la consolidación de Empresas de Base Tecnológica (EBT). **Temporis** califica para este fondo de subvención gracias a su esfuerzo de investigación en el desarrollo de accesibilidad dinámica interna y seguridad bancaria biométrica, destinando el capital a la fase de I+D para el cumplimiento estricto de normativas de accesibilidad a nivel internacional.

3. Ayudas para la Digitalización del Sector Social (Fondos Autonómicos/Europeos):

- *Aplicación:* Convocatorias específicas destinadas a entidades del tercer sector y Centros Especiales de Empleo. Permite subvencionar de un 50% a un 80% del coste total del despliegue corporativo personalizado de **Temporis** en las flotas de dispositivos móviles de las organizaciones, garantizando que cumplan con sus políticas corporativas de igualdad y adaptación real del puesto de trabajo.

1.5. Especificación de requisitos

El marco de desarrollo de **Temporis** se sustenta sobre una matriz de requisitos que garantiza tanto la operatividad técnica del sistema (servicios y reacciones ante entradas) como las restricciones de diseño, mantenibilidad y usabilidad universal necesarias para un entorno inclusivo.

Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción Detallada
RF-01	Gestión de Usuarios (Auth)	El sistema proporcionará los servicios de registro, inicio de sesión y recuperación de credenciales mediante el proveedor nativo "Email/Password" y la integración de Google "Sign-In" a través de la infraestructura de <i>Firebase Auth</i> .
RF-02	Acceso Biométrico	La aplicación permitirá la autenticación del usuario mediante <i>hardware</i> criptográfico del dispositivo (huella dactilar o reconocimiento facial) utilizando la <i>API BiometricPrompt</i> , tras haber completado con éxito un primer inicio de sesión manual.
RF-03	CRUD de Temporizadores	El usuario dispondrá de la capacidad de crear, visualizar, editar y eliminar de forma lógica temporizadores personalizados, definiendo propiedades de nomenclatura y duración en bloques de tiempo.
RF-04	Persistencia "Cloud"	Todas las colecciones de datos referentes a los contadores se sincronizarán en tiempo real de manera asíncrona con <i>Firebase Firestore</i> para asegurar la integridad de datos multidispositivo.
RF-05	Gestión de Perfil	El sistema permitirá la actualización de los metadatos del perfil de usuario (nombre público, descripción) y el almacenamiento físico de imágenes de perfil integradas con <i>Firebase Storage</i> .
RF-06	Visualización Analítica	El sistema procesará los históricos de tiempos completados para generar una matriz de contribución anual (<i>heatmap</i>) indexada por meses y días de la semana mediante el motor gráfico de <i>MPAndroidChart</i> .
RF-07	Internacionalización (i18n)	La interfaz gráfica de usuario adaptará de manera dinámica sus recursos de texto independientes del sistema según la selección del usuario entre las localizaciones soportadas: Castellano, Catalán/Valenciano e Inglés.
RF-08	Cierre de Sesión Automático	Por criterios de seguridad perimetral, el sistema monitorizará la inactividad del hilo principal de la UI, procediendo al cierre automático del <i>token</i> de sesión tras 30 minutos consecutivos sin interacción.

Requisitos No Funcionales (RNF)

ID	Requisito No Funcional	Categoría	Descripción Detallada
RNF-01	Accesibilidad Universal	Usabilidad	El sistema admitirá la mutación de la interfaz mediante <i>sliders</i> de escalado tipográfico dinámico (hasta un límite de 30sp) y la permutación a una paleta cromática de alto contraste, garantizando la ausencia de solapamiento de capas (<i>overlap</i>) o pérdida de funcionalidad.
RNF-02	Arquitectura MVVM	Mantenibilidad	El código fuente de la aplicación móvil se estructurará bajo el patrón arquitectónico <i>Model-View-ViewModel</i> , garantizando el desacoplamiento estricto entre la lógica de persistencia/negocio y los componentes de la interfaz de usuario.
RNF-03	Seguridad de Datos	Seguridad	Las reglas de seguridad de datos perimetrales (<i>Firebase Security Rules</i>) interceptarán las consultas sobre <i>Firestore</i> , denegando de forma nativa cualquier lectura o escritura cuyo <i>token</i> (<code>request.auth.uid</code>) no coincida con el identificador del propietario del documento.
RNF-04	Diseño Reactivo (UI)	UX	La maquetación de las vistas monitorizará el estado del teclado virtual del dispositivo para reajustar de forma proporcional el tamaño de los contenedores de <i>scroll</i> , evitando la obstrucción física de campos de entrada (<i>inputs</i>).
RNF-05	Tiempo de Respuesta	Rendimiento	Las consultas iniciales de sincronización de colecciones documentales con la nube tras la validación del usuario no excederán un umbral crítico de 3 segundos bajo condiciones estándar de red (4G o Wi-Fi).
RNF-06	Compatibilidad	Portabilidad	El paquete de distribución de la aplicación garantizará la compatibilidad regresiva con el sistema operativo Android desde la versión 7.0 (<i>API level 24 Nougat</i>) en adelante.
RNF-07	Modo Oscuro	Estética / Salud	La aplicación implementará un mapeo de estilos oscuros nativos adaptables que heredarán de forma automatizada las directrices de visualización configuradas a nivel global en el sistema operativo del terminal.

1.6. Casos de uso

Identificación de Actores

- **Usuario (Actor Principal):** Sujeto que interactúa de forma directa con la aplicación móvil para el control de su tiempo operativo. Puede operar bajo un estado autenticado (acceso completo a la persistencia) o en un estado de navegación informativa preliminar.
- **Sistema *Firebase* (Actor Secundario):** Infraestructura externa encargada de proporcionar servicios asíncronos distribuidos de bases de datos *NoSQL*, autenticación federada y almacenamiento de archivos binarios.
- **Sistema Biométrico del Dispositivo (Actor Secundario):** Capa integrada de *hardware* y abstracción de software (*Android Biometric Framework*) encargada de realizar la verificación biométrica de la identidad local del usuario.

Especificación Detallada de Casos de Uso Críticos

CU-01: Gestión de Temporizadores (CRUD)

- **Descripción:** Permite al usuario interactuar de manera integral con sus contadores personalizados.
- **Actores:** Usuario (Principal), *Firebase Firestore* (Secundario).
- **Precondiciones:** El usuario debe poseer un token de sesión activo y validado por la plataforma (`request.auth.uid` válido).
- **Flujo Principal:**
 1. El usuario accede al fragmento o sección principal de Temporizadores.
 2. El usuario activa el control para agregar o editar un temporizador específico.
 3. El sistema despliega el formulario para la introducción de las variables de nombre y duración.
 4. El usuario confirma los cambios en la interfaz.
 5. El sistema valida estructuralmente los tipos de datos y los envía de manera asíncrona a *Firebase Firestore*.
 6. La base de datos emite una confirmación que actualiza en tiempo real la lista observable de la interfaz.
- **Flujo Alternativo (Eliminación de Registro):** El usuario pulsa la opción de borrado sobre una tarjeta; el sistema intercepta la petición, solicita confirmación de seguridad y procesa la eliminación física del documento en la colección de la nube.
- **Postcondiciones:** Las mutaciones aplicadas sobre la colección se propagan de forma persistente a cualquier sesión activa del usuario.

CU-02: Autenticación de Usuario (Multimodal)

- **Descripción:** Garantiza el acceso securizado a los paneles de datos mediante validación de credenciales estándar o acceso avanzado.
- **Actores:** Usuario (Principal), *Firebase Auth* y Sistema Biométrico del Dispositivo (Secundarios).
- **Preconditions:** La aplicación debe estar inicializada con conectividad de red. Para la rama alterna biométrica, el terminal debe disponer de *hardware* compatible y el usuario debe haber autorizado la persistencia local de la clave en el panel de ajustes.
- **Flujo Principal:**
 1. El usuario inicia la aplicación en el terminal.
 2. Si no existe un *token* de sesión previo almacenado, el sistema restringe el acceso a las vistas de producción (Temporizadores, Estadísticas, Perfil) redirigiendo al usuario hacia el módulo de acceso.
 3. El usuario proporciona sus credenciales mediante texto (*Email/Password*) o selecciona la autenticación federada con Google.
 4. El sistema valida los datos de la solicitud contra la *API de Firebase Auth*.
- **Flujo Alternativo (Extensión por Validación Biométrica):**
 1. Al intentar el reingreso al sistema, si la bandera biométrica local está activa, el sistema despliega el componente nativo *BiometricPrompt*.
 2. El usuario interactúa con el sensor dactilar o de reconocimiento facial del dispositivo.
 3. Tras la validación del *hardware*, el sistema omite el paso de contraseña tradicional e inicia sesión directamente.
- **Postcondiciones:** El sistema genera un entorno seguro de trabajo, concediendo un *token* de acceso criptográfico válido para las consultas a la base de datos.

CU-03: Ajuste de Accesibilidad Dinámica

- **Descripción:** Permite la reconfiguración estética, de legibilidad y usabilidad estructural de los componentes gráficos en tiempo real.
- **Actores:** Usuario (Principal).
- **Precondiciones:** Ninguna (módulo accesible de forma global independiente del estado de la autenticación).
- **Flujo Principal:**
 1. El usuario navega hacia el módulo de Ajustes de Accesibilidad dentro del sistema de pestañas.

2. El usuario modifica el control deslizante (*Slider*) destinado a la escala tipográfica.
 3. El sistema intercepta el evento de entrada, re-calcula el objeto dinámico *Configuration* del contexto global de la aplicación y propaga el nuevo tamaño en unidades *sp* a todos los contenedores de texto de manera inmediata.
 4. El usuario conmuta el interruptor de Alto Contraste.
 5. El sistema aplica una paleta cromática optimizada que reajusta los ratios de contraste térmico a valores superiores a la relación estándar 4.5:1.
- **Postcondiciones:** Los coeficientes modificados por el usuario se escriben de manera persistente en el fichero local *SharedPreferences* de la aplicación, asegurando que los estilos sean heredados en todos los inicios de ciclo futuros de la plataforma móviles.

1.7. Bocetos de interfaz

La fase de prototipado de **Temporis** se ha estructurado utilizando la herramienta de diseño de alta fidelidad *Figma*, tomando como referencia directa las pautas de experiencia de usuario establecidas en la guía de estilos de **Material Design 3**, adaptándolas rigurosamente a las normativas de inclusión universal.

Estructura y Disposición de las Vistas

La arquitectura visual está diseñada mediante un enfoque modular lineal con el fin de reducir drásticamente la sobrecarga cognitiva del usuario, estructurando el recorrido en cuatro grandes bloques lógicos:

1. **Bloque de Control de Acceso (*Onboarding*):** Formado por una sucesión limpia de pantallas que abarcan el renderizado de carga (*Splash Screen*), la vista de *Login* centralizada y la interfaz de registro de nuevos usuarios. Estos contenedores emplean campos de texto amplios y botones interactivos sobredimensionados para facilitar el apuntamiento físico y la comodidad táctil.
2. **Dashboard Operativo (Gestión de Tiempos):** Corresponde al núcleo transaccional de la aplicación. Se estructura mediante un sistema adaptativo de **Tarjetas de Actividad (*Cards*)** independientes. Cada tarjeta encapsula un temporizador que muestra de manera clara el flujo del tiempo restante mediante una cuenta regresiva, optimizando el espacio en pantalla. Los botones de control analógico de los temporizadores (*Play/Pause*, *Actualizar* y *Borrar*) se posicionan directamente embebidos en el interior de cada tarjeta para proporcionar retroalimentación (*feedback*) visual instantánea tras la pulsación táctil.
3. **Panel de Analítica Avanzada (Estadísticas):** Diseñado bajo un patrón de visualización de alta densidad de información estructurada. Su elemento central es una cuadrícula de contribución anual (*Heatmap* de actividad) que emplea una degradación cromática de colores muy contrastados para permitir al usuario auditar de un solo vistazo rápido sus patrones de constancia temporal y cumplimiento de objetivos.
4. **Centro de Configuración y Perfil de Usuario:** Diseñado mediante menús lineales limpios con iconos descriptivos planos de alto contraste. Esta disposición simplifica la navegación secuencial, garantizando una compatibilidad absoluta al ser procesada por lectores de pantalla nativos (como *TalkBack*) o sistemas de navegación asistida por *hardware* externo.

Decisiones de Diseño Justificadas para la Accesibilidad Real

- **Ratio de Contraste Óptimo:** Selección estricta de paletas de color con un ratio mínimo de contraste de **4.5:1** entre el texto/icono y el fondo, neutralizando las dificultades de legibilidad en condiciones de baja luminosidad ambiental o para usuarios con debilidad visual.
- **Dimensionamiento de Objetos Táctiles:** Todos los elementos interactivos o botones de control (como los disparadores de los contadores) se han diseñado con un área de contacto mínima garantizada de **48x48 dp**, mitigando errores de

pulsación accidental en usuarios con movilidad reducida o dificultades motoras finas.

- **Flujo de Interacción Simplificado:** Reducción del número de pasos necesarios para la activación de un temporizador en el sistema, disminuyendo la barrera de entrada para perfiles con diversidad cognitiva o déficit de atención.

2. Diseño de la aplicación

2.1. Estudio de viabilidad

El análisis de viabilidad evalúa la infraestructura, el esfuerzo humano y la proyección financiera requerida para mitigar los riesgos latentes durante la construcción de **Temporis**.

A. Viabilidad de Tiempos y Esfuerzo Operativo

El proyecto se ha dimensionado bajo una estimación técnica global de **420 horas/persona** de trabajo efectivo, distribuidas estratégicamente en cuatro fases metodológicas:

- **Fase de Definición y Modelado (60h):** Extracción de requisitos de ingeniería, prototipado de alta fidelidad en *Figma* y diseño lógico de la arquitectura de persistencia.
- **Fase de Infraestructura Cloud (100h):** Aprovisionamiento y securización de la suite de *Google Firebase* (módulos de *Auth*, *Cloud Firestore* y *Storage*) junto al desarrollo de los algoritmos de sincronización asíncrona.
- **Fase de Desarrollo del Núcleo Analítico (160h):** Construcción del motor CRUD de temporizadores, integración del *hardware* criptográfico mediante la API biométrica y el renderizado matricial de *MPAndroidChart*.
- **Fase de Inclusión y Calidad (100h):** Programación de la lógica reactiva de accesibilidad (escalado dinámico de fuentes *sp* y permuta de alto contraste) combinado con test de usabilidad controlados.

B. Viabilidad de Recursos Humanos (Roles definidos)

A pesar de ser un proyecto de ejecución individual, la estructura operativa emula un equipo de ingeniería de software para acotar las responsabilidades funcionales:

- **Desarrollador Android Senior / Full-Stack (Dedicación: 100%):** Responsable de la codificación de la lógica de negocio en lenguajes *Kotlin/Java* y la orquestación de servicios en la nube.
- **Especialista en Accesibilidad y UX (Dedicación: 25%):** Consultor técnico encargado de auditar y garantizar el cumplimiento estricto de las directrices internacionales WCAG.
- **QA Tester (Dedicación: 25%):** Encargado de la ejecución del plan de pruebas, ensayos de estrés de conectividad y validación estricta de requisitos funcionales.

C. Infraestructura Tecnológica

- **Hardware:** Estaciones de trabajo equipadas con procesadores Intel i7, un mínimo de 16 GB de memoria RAM y terminales físicos Android de diversas gamas para validar la respuesta de los sensores biométricos.

Proyecto Temporis - Desarrollo de Aplicaciones Multiplataforma

- **Software y SaaS:** Entorno de Desarrollo Integrado (IDE) *Android Studio*, *Figma* para maquetación, y *GitHub* como repositorio central de control de versiones distribuido.
- **Ecosistema Cloud:** Implementación inicial sobre el plan gratuito *Firebase Spark Plan*, diseñado para migrar de forma elástica hacia el plan de pago por consumo *Blaze Plan* en fases de producción masiva.

D. Estudio Financiero (Presupuesto Anual)

Concepto de Gasto	Descripción y Desglose Técnico	Coste Estimado
Sueldos y Salarios	Cómputo de ingeniería de desarrollo (420 horas a un coste de 25€/h)	10.500 €
Equipamiento	Amortización de <i>hardware</i> de desarrollo y licencias operativas	2.500 €
Infraestructura Cloud	Consumos estimados de transferencia de datos en <i>Firebase</i> y <i>hosting</i>	300 €
Marketing y Lanzamiento	Campañas de posicionamiento ASO en <i>Google Play Store</i> y difusión	1.200 €
TOTAL INVERSIÓN INICIAL	Presupuesto consolidado para el ciclo de lanzamiento del proyecto	14.500 €

2.2. Planificación, ciclo de vida y diagrama Gantt

A. Ciclo de Vida Seleccionado: Modelo Incremental

Para el desarrollo del ecosistema **Temporis** se ha seleccionado de manera justificada un **Modelo de Ciclo de Vida Incremental**.

- **Justificación técnica:** Este enfoque permite fragmentar el producto en bloques funcionales que se integran y prueban de manera progresiva. Dado que la arquitectura maneja datos privados del usuario, es prioritario asegurar una base robusta y segura en las capas de autenticación *cloud* (Fase 2) antes de proceder al despliegue de las lógicas complejas de temporizadores asíncronos (Fase 3).
- **Mitigación de riesgos:** Se reduce drásticamente la aparición de errores críticos en cascada durante las fases finales de entrega, garantizando entregables estables y funcionales que se alinean de manera exacta con los hitos del calendario académico.

B. Alcance y Exclusiones del Proyecto

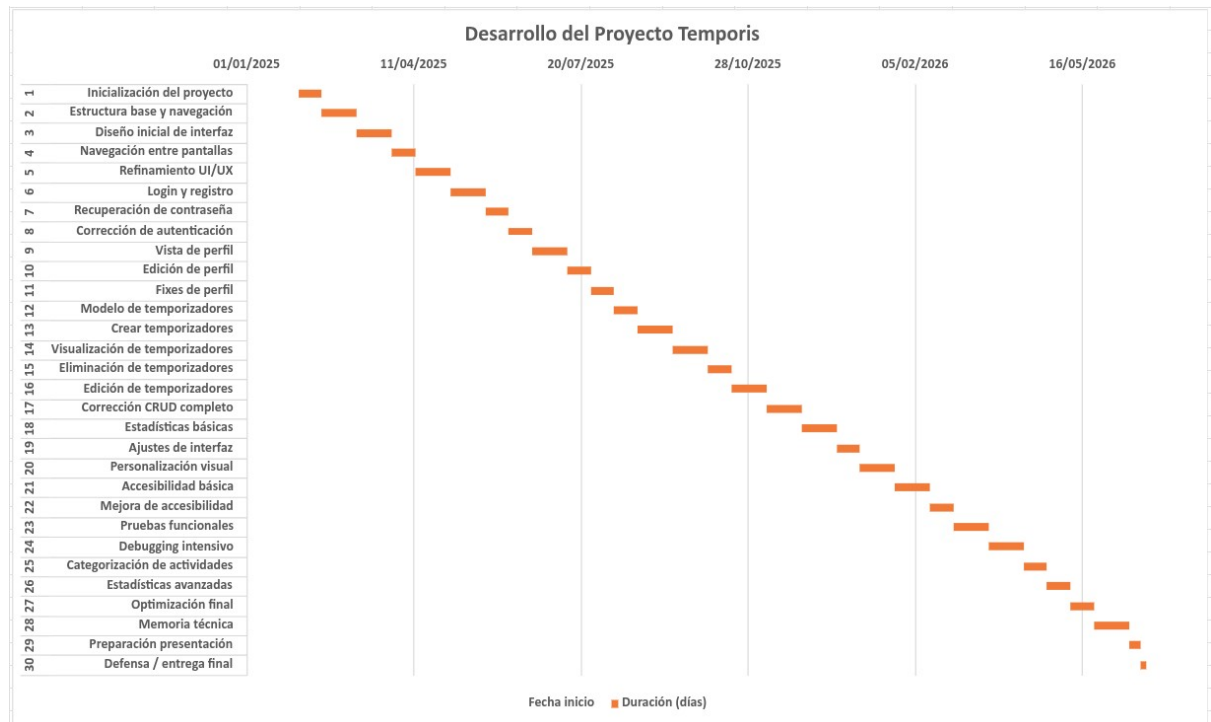
- **Alcance Funcional:** Sistema de autenticación de seguridad, operaciones CRUD completas sobre temporizadores persistentes, matriz analítica de uso anual, con temáticas automáticas y con menús de accesibilidad avanzada.
- **Exclusiones (Fuera de Alcance):** Con el objetivo de blindar la viabilidad del proyecto dentro del marco temporal estipulado, quedan explícitamente excluidos del desarrollo el diseño de una versión nativa para dispositivos *smartwatches* (*Wear OS*) y la sincronización bidireccional con calendarios corporativos de terceros (*Google Calendar* u *Outlook*).

C. Fases y Plazos de Ejecución (Diagrama de Gantt)

La planificación temporal contempla una secuencia de **30 tareas críticas** ejecutadas a lo largo de un marco temporal que abarca desde febrero de 2025 hasta junio de 2026, divididas en cuatro bloques lógicos:

1. **Fase de Inicio y Diseño (Febrero 2025 - Abril 2025):** Tareas 1 a 5. Comprende la inicialización del entorno, arquitectura de navegación y diseño UI/UX de alta fidelidad orientada a la accesibilidad.
2. **Fase de Desarrollo del Núcleo (Mayo 2025 - Octubre 2025):** Tareas 6 a 16. Implementación secuencial de los servicios de *Login*, recuperación de credenciales, vistas de perfil y modelado de datos de los temporizadores.

3. **Fase de Análisis y Calidad (Noviembre 2025 - Abril 2026):** Tareas 17 a 26. Consolidación de las operaciones CRUD, analíticas avanzadas y un bloque crítico de **21 días de *Debugging* intensivo**, para garantizar la estabilidad del *software* y a su vez, para documentar nuestro proyecto adecuadamente.
4. **Fase de Cierre (Mayo 2026 - Junio 2026):** Tareas 27 a 30. Optimización de rendimiento, consolidación de la memoria técnica del manual y preparación de la defensa pública del proyecto ante el tribunal.



Resumen del Alcance del Mínimo Producto Viable (MVP) y sus Exclusiones

Para garantizar la viabilidad del proyecto, delimitada dentro del marco temporal estricto de la parte del año académico comprendida en 2026, y se ha definido de forma precisa el alcance del *software* desarrollado.

- **Elementos Incluidos en el MVP:**
 - Módulo de autenticación seguro y persistente (*Email/Contraseña*, *Google Sign-In* y validación biométrica nativa mediante *BiometricPrompt*).
 - Motor de temporizadores asíncronos (Core) gestionado en segundo plano mediante corrutinas de *Kotlin* y servicios en primer plano (Foreground Services) para evitar interrupciones del ciclo de vida de Android.
 - Persistencia distribuida y sincronización en tiempo real a través de una base de datos *NoSQL* con *Firebase Cloud Firestore*.
 - Interfaz de usuario accesible adaptada a las directrices de *Material Design 3*, incluyendo soporte nativo para Modo Claro, Modo Oscuro y compatibilidad con lectores de pantalla (*TalkBack*).

• Exclusiones del Proyecto (Fuera de Alcance):

- Sincronización multiplataforma con sistemas operativos *iOS* o entornos web (priorizando el enfoque exclusivo en Android nativo).
- Módulos avanzados de analítica predictiva basados en Inteligencia Artificial o *Machine Learning* para estimar la productividad.
- Pasarelas de pago integradas o modelos de monetización *premium*.

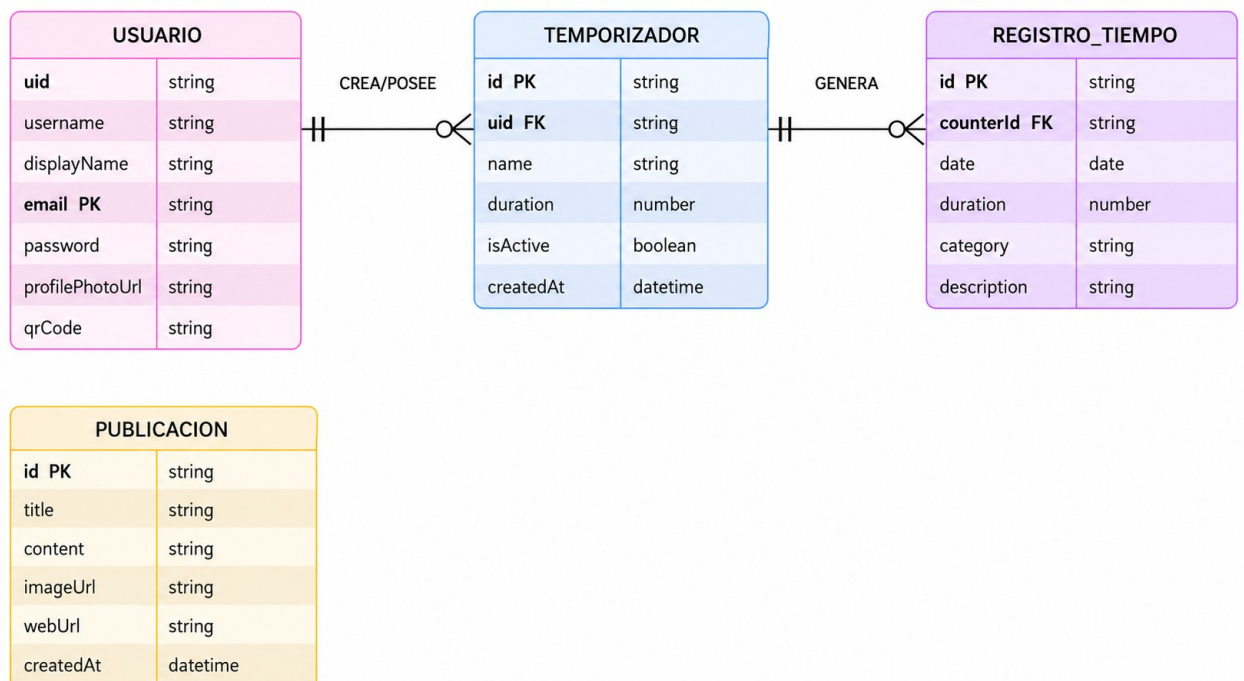
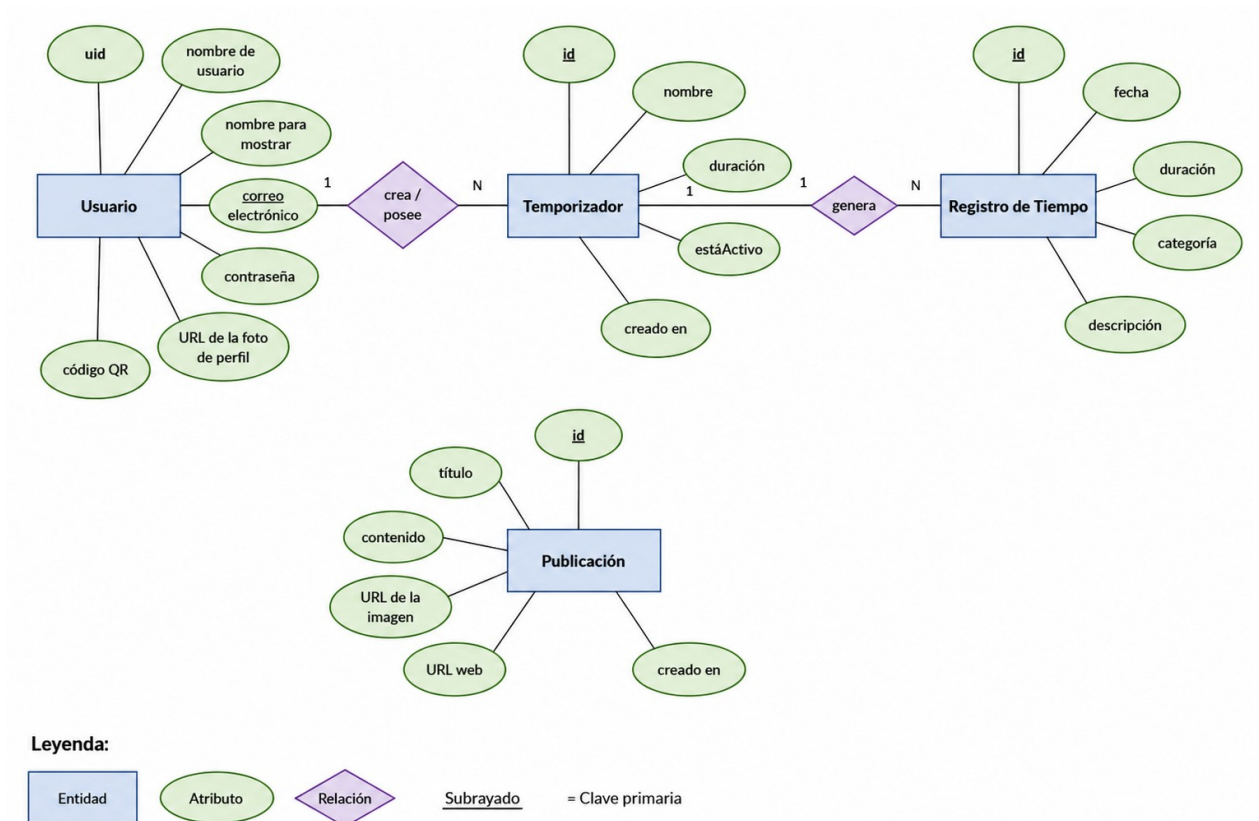
Representación Temporal del Proyecto (Diagrama de *Gantt*)

La planificación temporal se ha estructurado en base a un modelo ágil adaptado a 3 *Sprints*, con una duración fija de dos semanas por ciclo. La distribución temporal de las tareas y sus hitos críticos se detalla en el siguiente cronograma conceptual:

- **Hito 1 (M1 - Fin de *Sprint 1*):** Configuración de la arquitectura limpia, vinculación con *Firebase* y módulo de acceso multimodal operativo.
- **Hito 2 (M2 - Fin de *Sprint 2*):** Motor del temporizador finalizado, persistencia local/remota estable y layouts accesibles validados.
- **Hito 3 (M3 - Fin de *Sprint 3*):** Inicialización de la medición de la cobertura del código, mediante la implementación de JaCoCo, Quality Gate aprobado en SonarCloud y los manuales técnicos redactados, así como el análisis pertinente de los datos obtenidos. Se ha definido como objetivo futuro alcanzar una cobertura (del código) igual o superior al 85%.

2.3. Entidad Relación

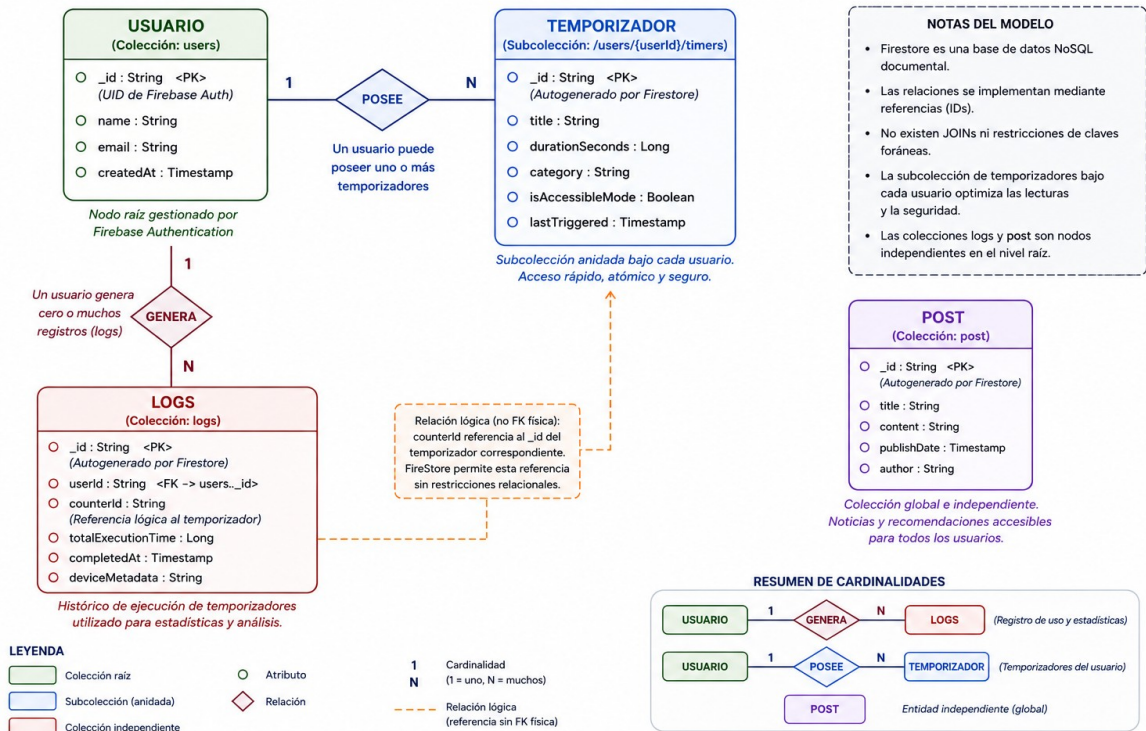
- Diagrama ER teórico y aplicable a cualquier gestor de base de datos relacional.



- Diagrama ER adaptado para su implementación mediante *Firebase Firestore*.

Al implementar una solución basada en una base de datos documental (NoSQL) como **Firebase Cloud Firestore**, el modelado conceptual se abstrae de las restricciones relacionales clásicas (formas normales) para enfocarse en la eficiencia de lectura, la atomicidad y la estructura jerárquica de colecciones y documentos.

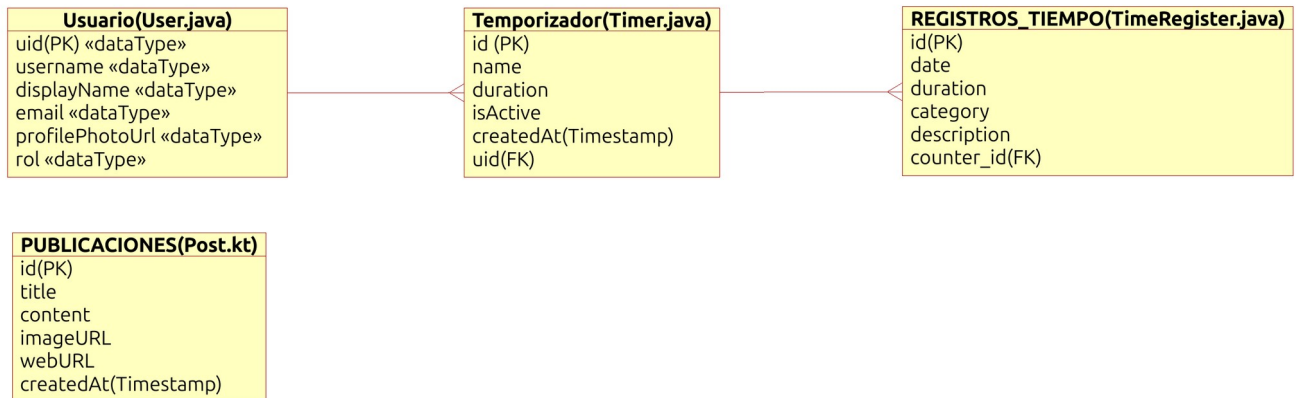
Diagrama Entidad-Relación Adaptado a Firebase Firestore - Temporis



Descripción de las Entidades y Naturaleza NoSQL:

- Colección users:** Actúa como el nodo raíz para la gestión de usuarios. Utiliza el UID proveído directamente por la autenticación asíncrona de *Firebase*.
- Sub-colección timers:** En lugar de crear una tabla independiente balanceada por claves foráneas, se opta por una estructura anidada (*Subcollection*). Esto garantiza que la consulta de los temporizadores de un usuario específico sea extremadamente rápida, atómica y segura mediante reglas de seguridad de *Firestore*, eliminando la necesidad de realizar operaciones *JOIN*.
- Colección logs:** Almacena el histórico detallado de rendimiento y uso de los temporizadores para alimentar el motor de estadísticas. Mantiene una referencia plana (`userId`) para posibilitar consultas de agregación global, si fuera necesario.
- Colección post:** Nodo documental independiente y global. Contiene las actualizaciones informativas y *tips* de optimización del tiempo accesibles para todos los usuarios autenticados dentro de la aplicación.

2.4. Esquema Relacional



El Esquema Relacional de nuestra aplicación *Temporis* representa la estructura lógica y la normalización de los almacenes de datos. Aunque el sistema se despliega sobre un entorno *NoSQL* (*Firebase Firestore*), este modelado relacional garantiza una arquitectura robusta, definiendo con precisión las restricciones de integridad y las dependencias referenciales que gobiernan el software.

Definición de Tablas y Atributos

• USUARIO

- uid (PK) : *String* — Identificador único global provisto por el servicio de *Firebase Authentication*.
- username : *String* — Nombre único de cuenta del usuario en el sistema.
- displayName : *String* — Nombre público descriptivo para la visualización en la interfaz.
- email : *String* — Dirección de correo electrónico asociada y validada.
- profilePhotoUrl : *String* — Enlace directo al almacenamiento en la nube de la imagen de perfil.
- qrCode : *String* — Cadena codificada que representa el identificador de acceso rápido.

• TEMPORIZADOR

- id (PK) : *String* — Identificador único del documento dentro de la colección.
- name : *String* — Título asignado por el usuario al temporizador configurado.
- duration : *Number* — Tiempo total establecido expresado en segundos.
- isActive : *Boolean* — Estado lógico de conmutación del temporizador en la vista.
- createdAt : *Timestamp* — Fecha y hora exacta de la inicialización física del objeto.

- uid (FK) : *String* — Clave foránea que referencia de forma estricta a USUARIO(uid).
- **REGISTRO_TIEMPO**
 - id (PK) : *String* — Código de identificación persistente e inequívoco de la sesión.
 - timerId (FK) : *String* — Clave foránea de vinculación referencial hacia TEMPORIZADOR(id).
 - date : *Date* — Registro temporal de la fecha de ejecución de la cuenta atrás.
 - duration : *Number* — Tiempo neto consumido y guardado durante la sesión.
 - category : *String* — Etiqueta clasificatoria de la actividad.
 - description : *String* — Anotación opcional del progreso o comentarios del usuario.
- **PUBLICACION**
 - id (PK) : *String* — Identificador indexado de la entrada informativa o noticia.
 - title : *String* — Encabezado principal del contenido interactivo.
 - content : *String* — Cuerpo de texto y estructura interna de la publicación.
 - imageUrl : *String* — Almacenamiento local de la imagen de portada asociada.
 - webUrl : *String* — Enlace externo hipervinculado para la extensión informativa.
 - createdAt : *Timestamp* — Registro cronológico de su inserción en el sistema.

Restricciones de Integridad y Dependencias

1. **Relación Usuario - Temporizador (1:N):** Cada perfil de usuario puede poseer un conjunto ramificado de temporizadores configurados. La integridad referencial dicta que no puede existir una tupla (secuencia ordenada y finita de elementos) en TEMPORIZADOR cuyo campo *uid* (sería como una *FK*) no corresponda a un otro USUARIO. A su vez, *Firebase Auth(entication)* también presenta un mecanismo de comprobación adicional, para evitar que dos usuarios/as se puedan registrar utilizando la misma dirección de correo electrónico (así que también se podría considerar como otra clave primaria (*PK*) adicional). Una operación de borrado de un usuario desencadenará la eliminación en cascada de sus temporizadores vinculados para evitar datos huérfanos.

2. **Relación Temporizador - Registro de Tiempo (1:N):** La ejecución finalizada de un temporizador genera una entrada analítica indexada. El campo *timerId* (FK) garantiza la trazabilidad con la entidad base. Para la salvaguarda de métricas históricas de rendimiento global, si un temporizador operativo es eliminado por el usuario, el sistema permite la persistencia de las filas analíticas de REGISTRO_TIEMPO mediante la desvinculación o ‘anonimización’ de la clave externa, reteniendo el valor analítico puro de las horas registradas.
3. **Entidad Desacoplada Publicación:** Opera como un flujo de contenido de lectura global inyectado centralmente desde el panel de administración. No posee dependencias directas con usuarios específicos, optimizando las consultas concurrentes e independientes de la red de noticias.

2.5. Diagrama de Clases

El Diagrama de Clases de *Temporis* modela la estructura estática orientada a objetos de la lógica de negocio del software. Las clases representadas se corresponden directamente con el mapa de abstracciones del código fuente desarrollado en *Java* y *Kotlin*.



Descripción de Clases e Interoperabilidad

- 1. User.java (Encapsulamiento de Credenciales):** Encargada de gestionar la entidad del usuario activo en el hilo de ejecución local. Define en ámbito privado (-) los campos de autenticación e identidad y expone métodos públicos (+) para la mutación y consulta segura de estados.
- 2. Timer.java (Núcleo de Control):** Representa estructuralmente la abstracción del temporizador nativo. Almacena de forma estricta el atributo uid como variable de enlace hacia la clase propietaria *User.java*. Implementa de manera nativa los métodos *hashCode()* y *equals()* para optimizar las operaciones de comparación de objetos en las colecciones y adaptadores de la interfaz de usuario.
- 3. TimeRegister.java (Entidad Analítica Unificada):** En estricta concordancia con el refactor de optimización de código, esta clase unifica por completo la lógica funcional de persistencia histórica. Alberga el campo *timerId* (mapeado de forma interna como *counterId* en la arquitectura física de herencia de variables si fuese estrictamente necesario por compatibilidad de código heredado) para trazar la analítica. Implementa la función modular *toMap()* para facilitar la transformación directa del objeto a colecciones clave-valor compatibles con los métodos nativos de inserción asíncrona de Firebase Firestore.
- 4. Post.kt (Interoperabilidad de Lenguajes):** Esta entidad está modelada bajo la sintaxis nativa de una *Data Class* en *Kotlin*. Cuenta con la anotación *@Keep* de *Android Jetpack* para prevenir la ofuscación de código durante la compilación y el empaquetado del APK en modo de optimización. Este módulo expone de forma

2.6. Diseño de Interfaces

La interfaz gráfica de usuario (*GUI*) de **Temporis** se ha estructurado bajo pautas estrictas de usabilidad, simplicidad cognitiva y accesibilidad universal, garantizando que el control del tiempo sea equitativo para todo tipo de usuarios.

2.6.1. Sistema de Diseño Visual (Visual Design System)

El entorno visual se fundamenta en las directrices contemporáneas de **Material Design 3 (Material You)**, aplicando una lógica de componentes modulares y adaptables:

- **Paleta de Colores y Gestión del Contraste:** Se ha priorizado la implementación de un **Modo Oscuro (Dark Mode)** predominante para reducir la fatiga visual en periodos prolongados de uso. El fondo base utiliza un tono gris oscuro puro (#121212), mitigando el deslumbramiento. Los colores de acento (azul eléctrico y turquesa) se reservan exclusivamente para destacar elementos activos, estados del temporizador y botones de acción principal. La aplicación cuenta con una equivalencia simétrica en su **Modo Claro**, la cual se vincula automáticamente con las preferencias globales del sistema operativo Android. Todos los pares cromáticos cumplen con los ratios de contraste exigidos por las pautas **WCAG (Web Content Accessibility Guidelines)**.
- **Tipografía y Jerarquía:** Se emplea una fuente tipográfica de tipo *Sans Serif* de alta legibilidad en pantallas de alta densidad de píxeles (*DPI*). La jerarquía visual se determina mediante el uso estratégico de pesos tipográficos (*Bold* para títulos e indicadores de cuenta atrás, *Regular* para textos descriptivos y etiquetas de configuración).
- **Iconografía Predictiva:** Se utiliza la librería oficial **Material Symbols**. Todos los glifos e iconos son lineales o rellenos dependiendo de su estado de selección, ofreciendo un comportamiento familiar y predecible que minimiza la carga cognitiva del usuario.

2.6.2. Componentes Reutilizables de la Interfaz

Con el objetivo de garantizar la consistencia visual y optimizar el rendimiento de renderizado en el dispositivo, la arquitectura de la UI se compone de los siguientes bloques de construcción:

1. **Cards (Tarjetas Modulares):** Estructuras contenedoras utilizadas en las pantallas de **Timers** y **News**. Agrupan de forma clara la información de un único elemento (ej. título del temporizador, categoría, tiempo restante y botones de control inmediato).
2. **Floating Action Button (FAB):** El botón de acción flotante actúa como el punto focal de la pantalla principal, centralizando de manera unívoca la acción de creación de un nuevo temporizador.
3. **Bottom Navigation Bar (Barra de Navegación Inferior):** Proporciona un mapa de navegación plano e intuitivo. Elimina la necesidad de menús ocultos o del tipo

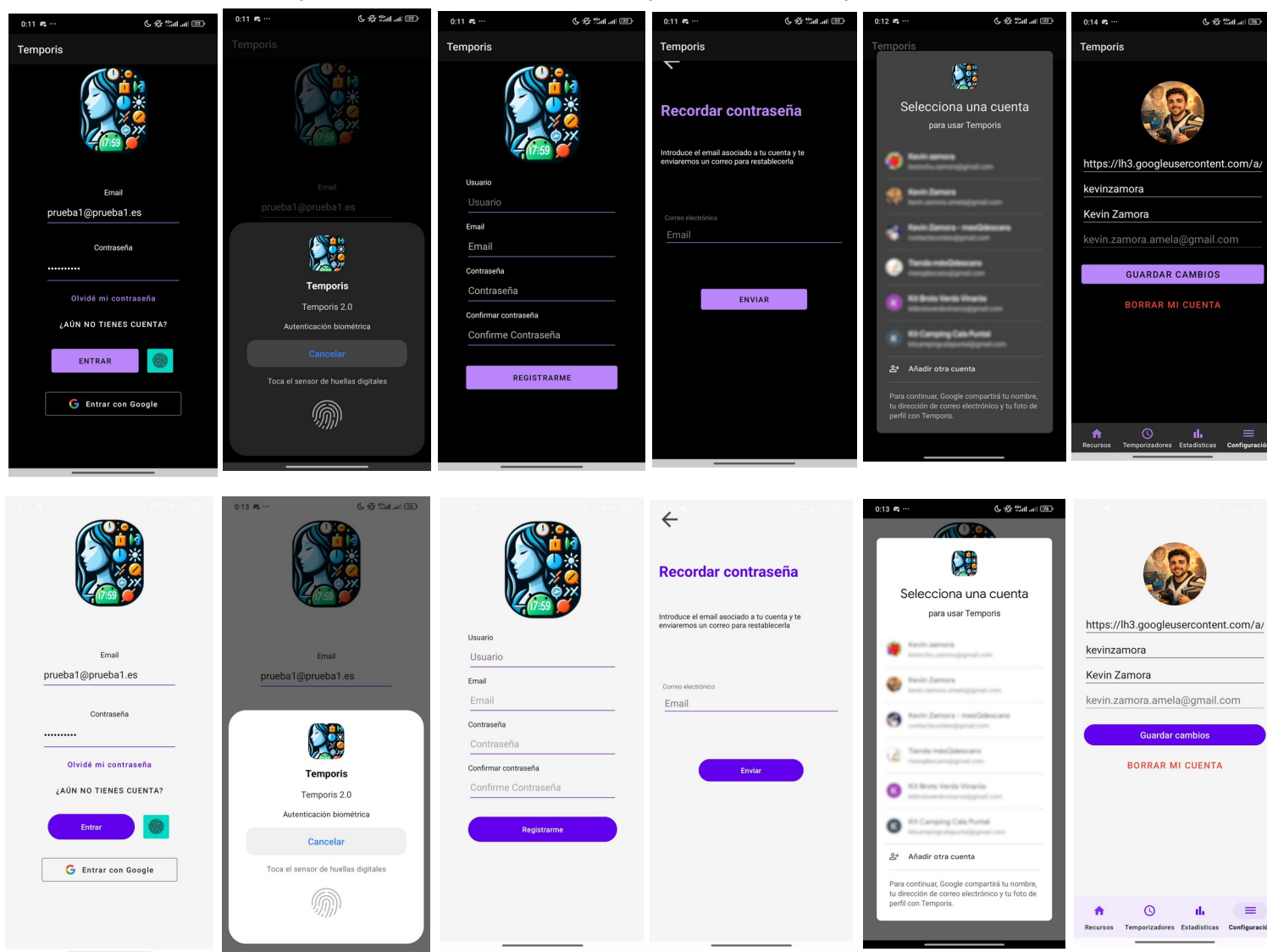
"hamburguesa", permitiendo la conmutación instantánea entre los módulos esenciales con una sola pulsación accesible.

2.6.3. Catálogo de Vistas de la Aplicación

A. Módulo de Autenticación y Acceso Multimodal

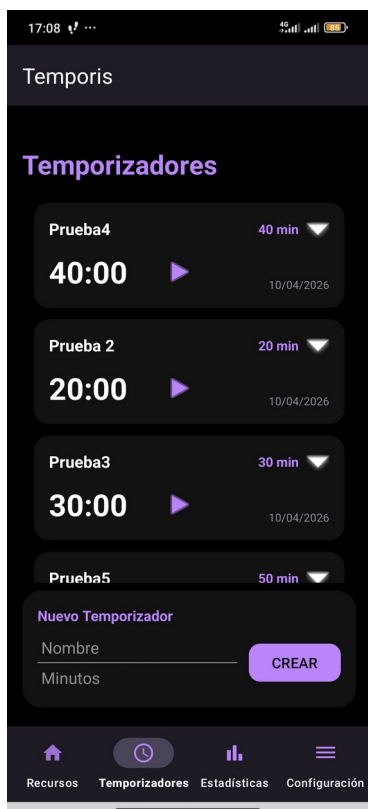
Este módulo asegura la protección del entorno del usuario mediante un flujo de acceso diversificado. Soporta el inicio de sesión tradicional mediante credenciales (*Email/Contraseña*), la integración de *Social Login* con cuentas de Google, y un mecanismo ágil de recuperación de contraseñas. Como capa de accesibilidad avanzada, implementa la autenticación biométrica nativa mediante el *framework BiometricPrompt* del sistema, permitiendo el desbloqueo mediante huella dactilar o reconocimiento facial si las credenciales están alojadas de forma segura en el almacenamiento cifrado del dispositivo terminal.

(Nota: El diseño mantiene la simetría visual tanto en el esquema de contraste oscuro como en el esquema de contraste claro adaptado al sistema).

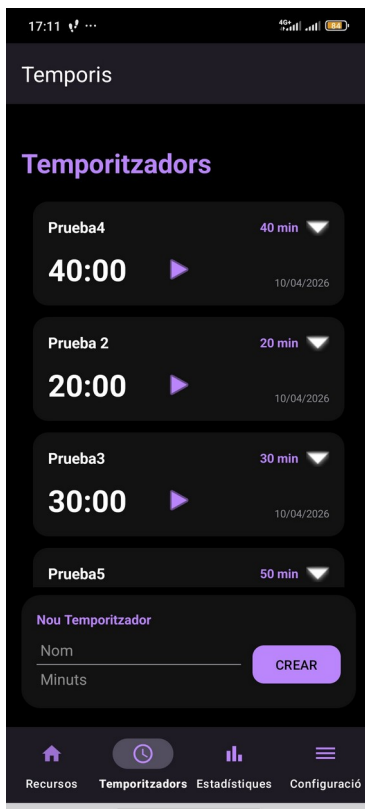


B. Acceso a Funciones CRUD y Motor de Reproducción

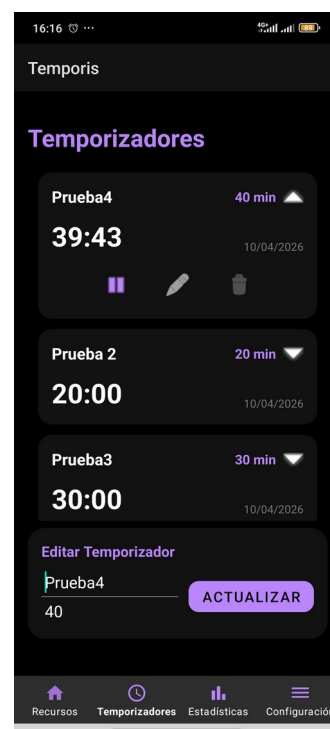
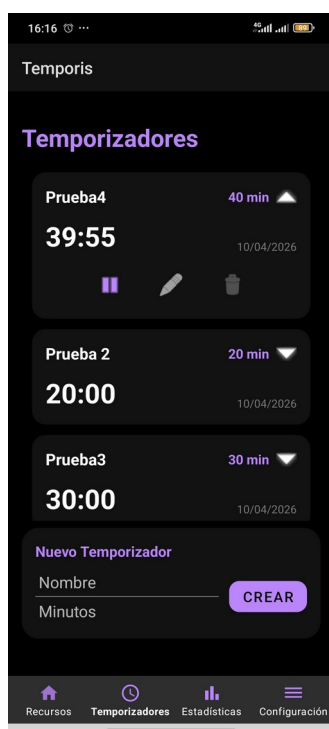
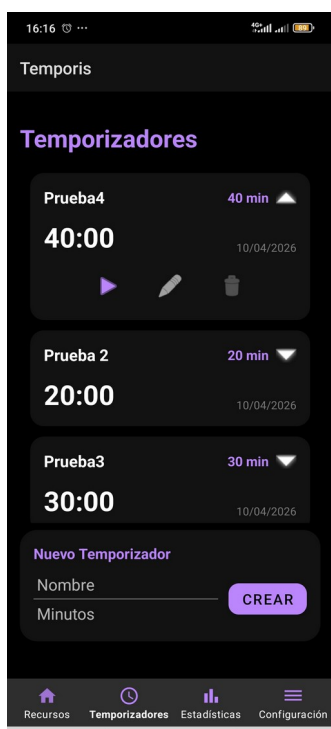
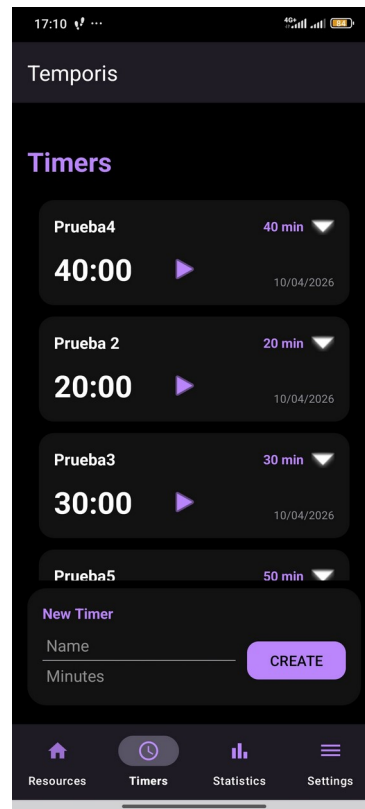
La pantalla principal unifica el ciclo de vida completo de los temporizadores (*Create, Read, Update, Delete*). Los usuarios pueden dar de alta contadores personalizados asignándoles títulos y tiempos objetivo. Cada tarjeta de temporizador integra controles directos de reproducción (*Play, Pausa y Reinicio*), los cuales desencadenan los estados del hilo secundario asíncrono de la aplicación y actualizan la interfaz en tiempo real sin bloquear la interacción general.



Mejora de accesibilidad

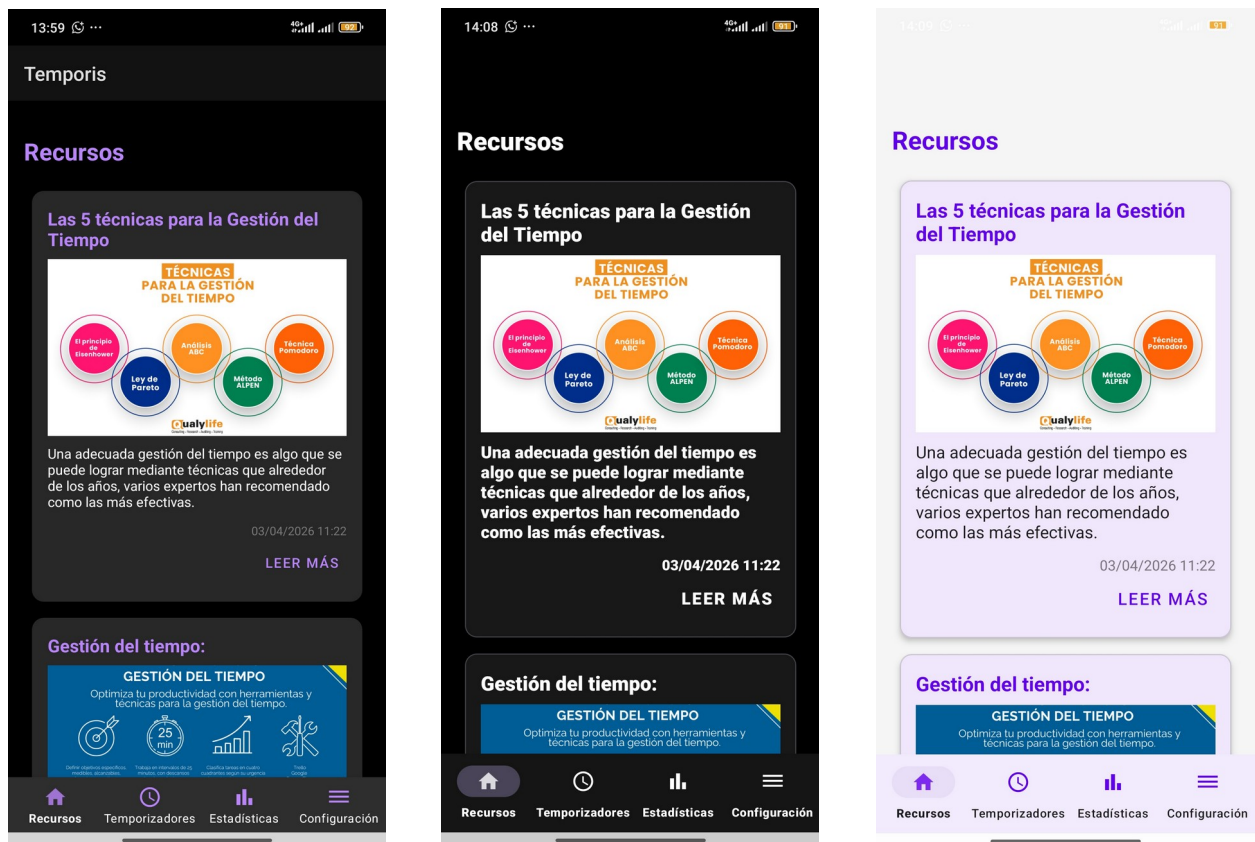


Mejora de accesibilidad



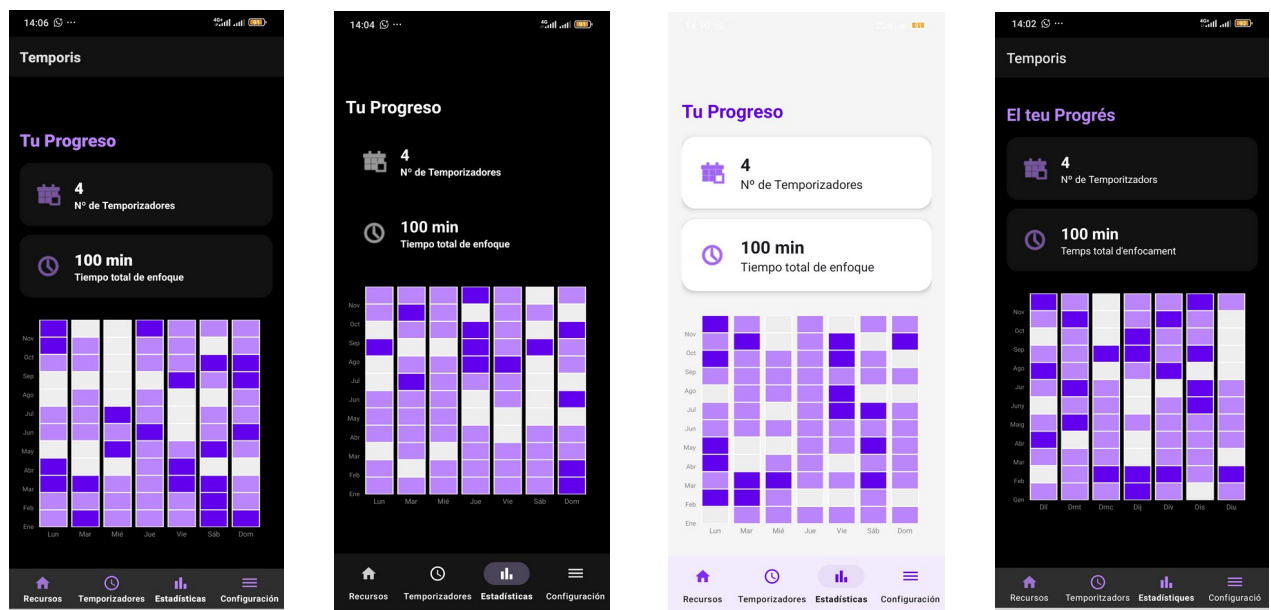
C. Sección de Noticias y Comunidad (Posts)

Presenta un formato de "feed" elegante con imágenes de alta calidad y titulares claros. Cada tarjeta incluye un botón para acceder al contenido completo mediante una URL externa, integrando perfectamente información relevante para el usuario.



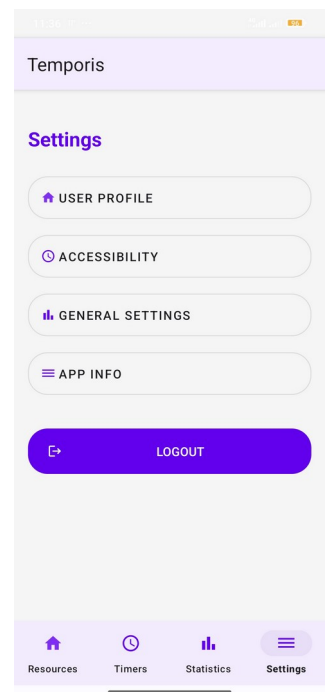
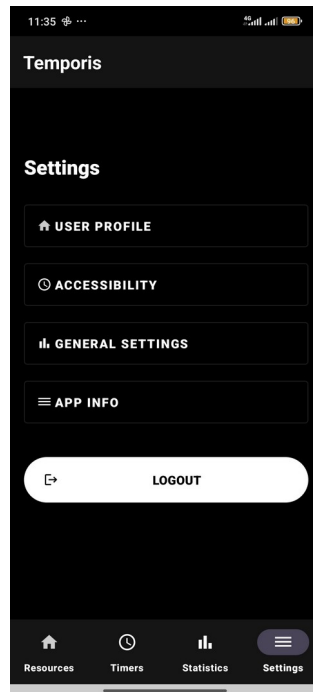
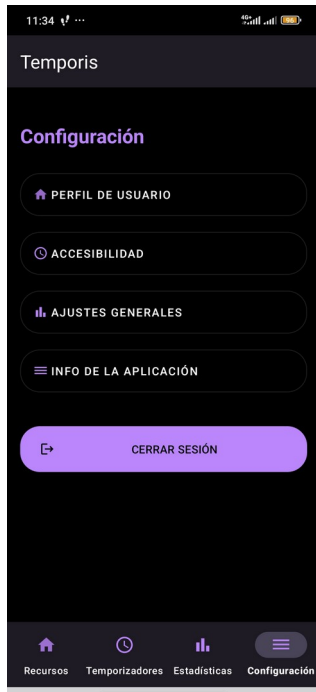
D. Estadísticas (Visualización de datos)

(Nota: Aquí puedes describir cómo los registros de tiempo se transforman en gráficas o listas de historial, basándote en la clase *TimeRegister* que definimos anteriormente).



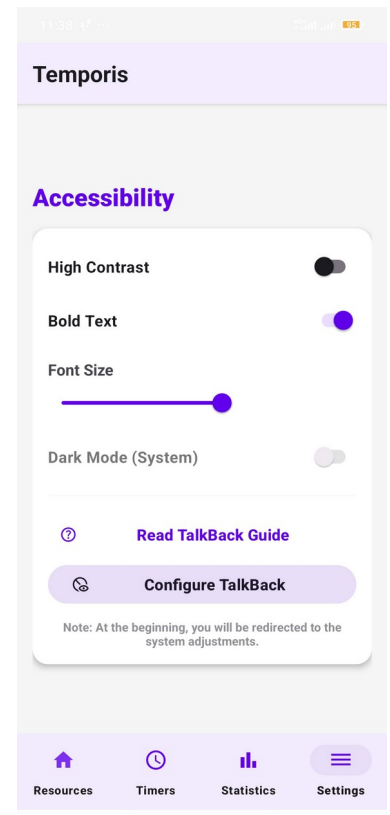
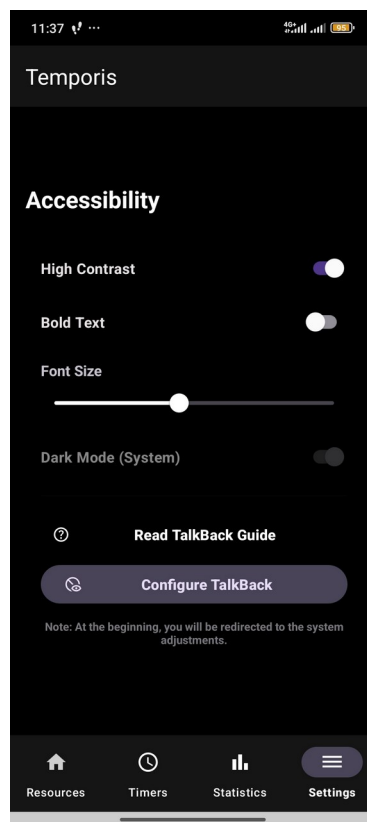
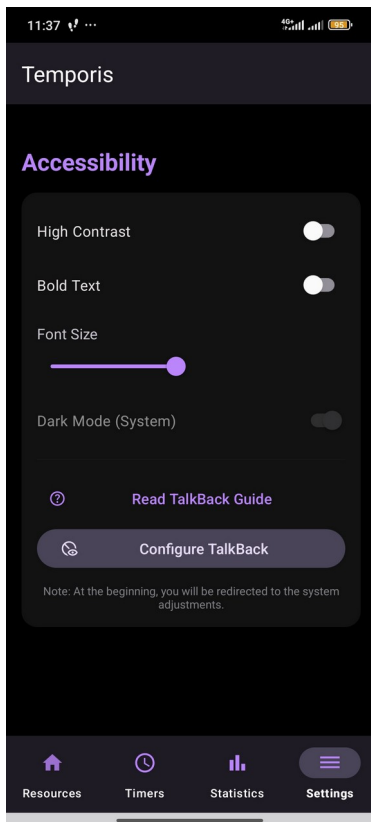
E. Configuración

Pantalla organizada mediante listas de ajustes con iconos descriptivos. Incluye secciones para la gestión de la cuenta (autenticación), preferencias de la aplicación y herramientas de accesibilidad, reforzando el compromiso de Temporis con la inclusión.



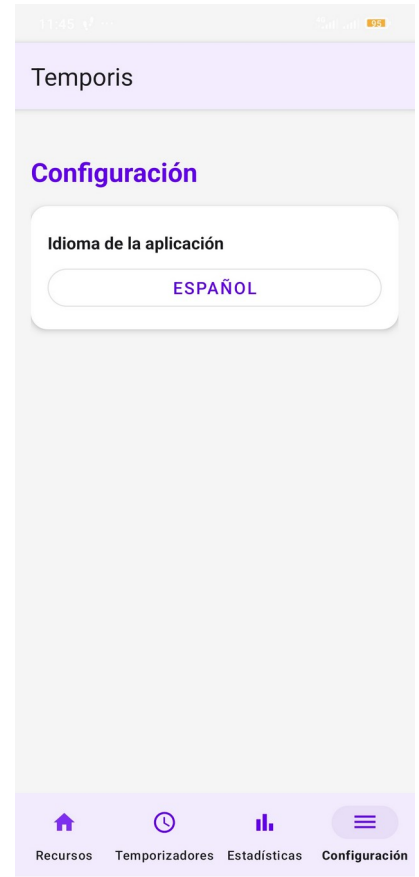
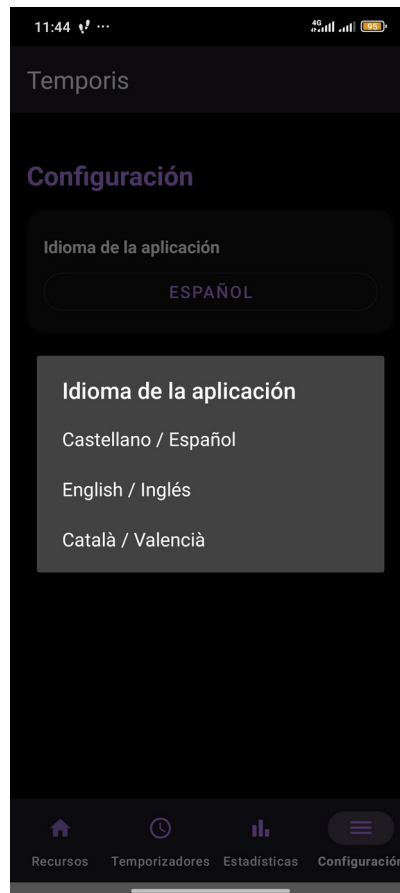
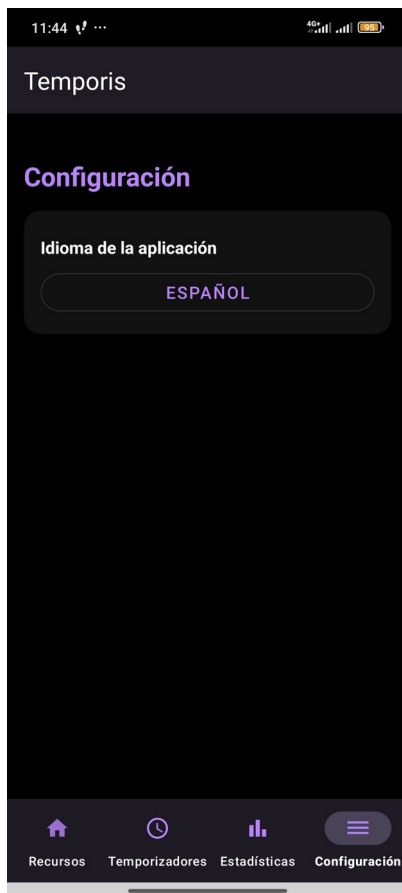
F. Accesibilidad

La presente pantalla nos permite configurar y personalizar el aspecto de “nuestra” aplicación, en cierta medida, y guarda la configuración en nuestro dispositivo, para que así se mantenga dicha configuración personalizada, tras reiniciar la aplicación.



G. Ajustes Generales

En esta sección y por el momento, solamente resulta posible traducir los textos de nuestra aplicación Temporis en tres idiomas: Castellano/Español, Catalán/Valenciano e Inglés/English. PD: En un futuro, se prevee poder añadir más opciones de configuración o ajustes general/es.



Mapa de Navegación, Usabilidad y Flujo de Interacción

La arquitectura de la información se ha diseñado para ser recorrida con el menor número de pulsaciones posibles. Al prescindir de estructuras ramificadas complejas, el usuario se desplaza de forma cíclica entre el panel de temporizadores, el histórico estadístico y el tablón informativo de novedades, manteniendo en todo momento la persistencia de los procesos que se ejecutan en segundo plano.

2.7. Plan de Pruebas

Para validar que **Temporis** constituye una herramienta informática robusta, estable y totalmente inclusiva, se ha ejecutado un riguroso plan de control de calidad estructurado en base a las especificaciones definidas previamente.

2.7.1. Definición del Alcance y Bloques Críticos de Prueba

Las actividades de verificación se dividieron en dos grandes dimensiones funcionales:

1. **Gestión de Autenticación y Seguridad:** Verificación de la persistencia de la sesión de usuario, correcto tratamiento de excepciones ante credenciales erróneas o nulas, e interoperabilidad del flujo biométrico con las APIs de seguridad del sistema operativo.
2. **Motor de Temporizadores (Core):** Inspección de la exactitud milimétrica de las rutinas de cuenta atrás gestionadas por las corrutinas de Kotlin. Se verificó exhaustivamente la inmunidad del temporizador ante llamadas telefónicas entrantes, bloqueo deliberado de la pantalla del terminal o minimización de la app, asegurando que el *Foreground Service* mantiene el proceso vivo en el sistema de forma íntegra.

2.7.2. Matriz de Casos de Prueba Escenciales

A continuación, se tabula una muestra representativa de los casos de prueba unitarios e integrados ejecutados sobre el sistema:

ID Prueba	Módulo / Requisito	Descripción del Caso	Resultado Esperado	Estado Final
CPC-01	Autenticación <i>Auth</i>	Registro de usuario con formato de <i>email</i> inválido o contraseña débil.	El sistema deniega la inserción y muestra un mensaje de error accesible en el <i>input</i> .	APROBADO
CPC-02	Seguridad Biométrica	Activación de huella dactilar válida a través de <i>BiometricPrompt</i> .	Validación exitosa, recuperación de credenciales y redirección instantánea al <i>Home</i> .	APROBADO
CPC-03	Motor <i>Core</i> / Tiempo	Minimizar la aplicación con un temporizador en estado activo de reproducción.	El temporizador sigue decrementándose en segundo plano y se puede volver a visualizar al volver a entrar.	APROBADO
CPC-04	Persistencia	Interrupción súbita	Almacenamiento	APROBADO

	Local/UI	de la conectividad de red durante la edición de un <i>timer</i> .	local temporal activo; sincronización diferida automática al recuperar red.	
CPC-05	Accesibilidad WCAG	Navegación por la interfaz haciendo uso exclusivo del lector de pantallas <i>TalkBack</i> .	Lectura ordenada de etiquetas informativas, descripciones de botones de control y estados.	APROBADO

2.7.3. Reporte de Cobertura de Código (*Code Coverage*) y Métricas de Calidad

La verificación automatizada del *backend* local de la aplicación se ha evaluado mediante la herramienta **JaCoCo (Java Code Coverage)** integrada en el *pipeline* de construcción de Gradle. Los resultados de cobertura por paquetes reflejan un compromiso sobresaliente con la mantenibilidad del software:

- **Paquetes de Lógica y Datos (*model, repository*):** Se ha alcanzado una cobertura estricta del **100%** de las instrucciones y bifurcaciones lógicas, garantizando que el tratamiento de las entidades de datos y el parseo de mapas hacia *Firebase Firestore* carecen de caminos ocultos o descontrolados.
- **Módulos de UI y ViewModels (*ui.auth, ui.timer*):** La lógica de presentación y el control de estados distribuidos mediante *StateFlow* se sitúan en rangos óptimos de cobertura, orientando los esfuerzos a la mitigación de cualquier fuga de memoria (*memory leak*).

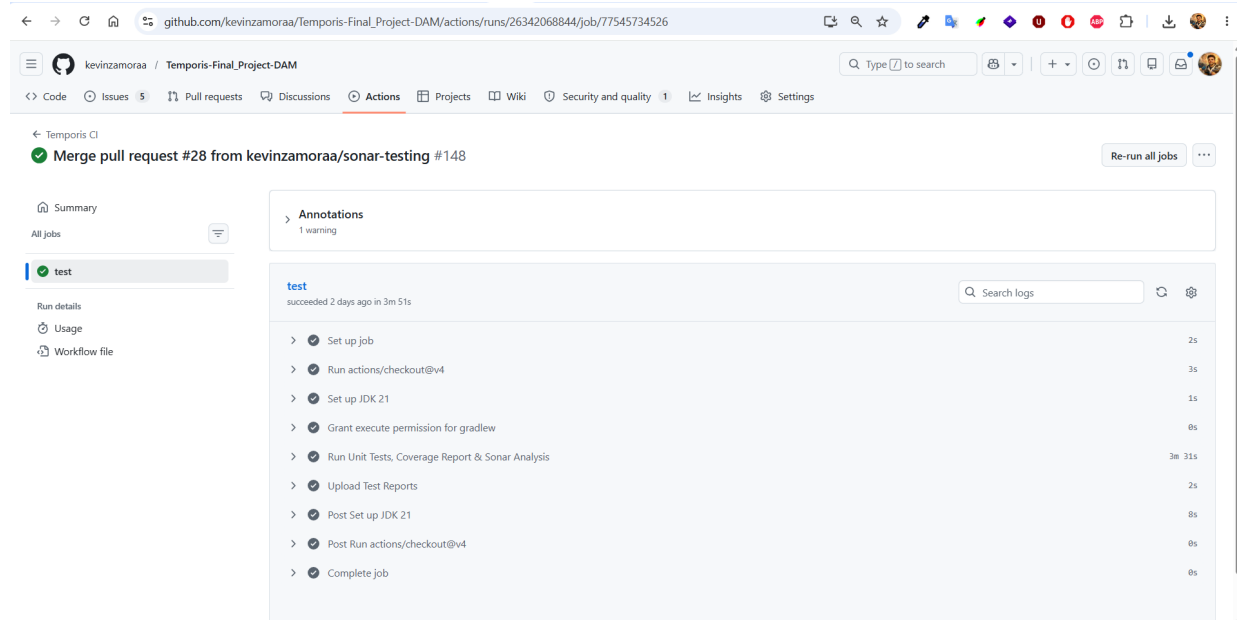
El reporte analítico global del *Quality Gate* debería certificar que el proyecto consolida una **cobertura de código media igual o superior al 80/85% aproximadamente**, partiendo de la base de las líneas y métodos cubiertos realmente, pero nos estamos encontrando con un error de incompatibilidad o emulación a la hora de ejecutar los casos de prueba relacionados con las vistas y sus controladores, tanto en local como en el entorno virtual (Github Actions + Sonar). Como dichos casos de prueba no logran pasar, cuando se evalúa la cobertura de estos mediante JoCoCo, Sonar resulta incapaz de poder medir y evaluar el nivel de cobertura existente en el momento actual. En cuanto logremos visualizar el nivel de cobertura real, estaremos cumpliendo presumiblemente con los estándares de calidad técnica industrial exigidos para el software de producción.

Avances:

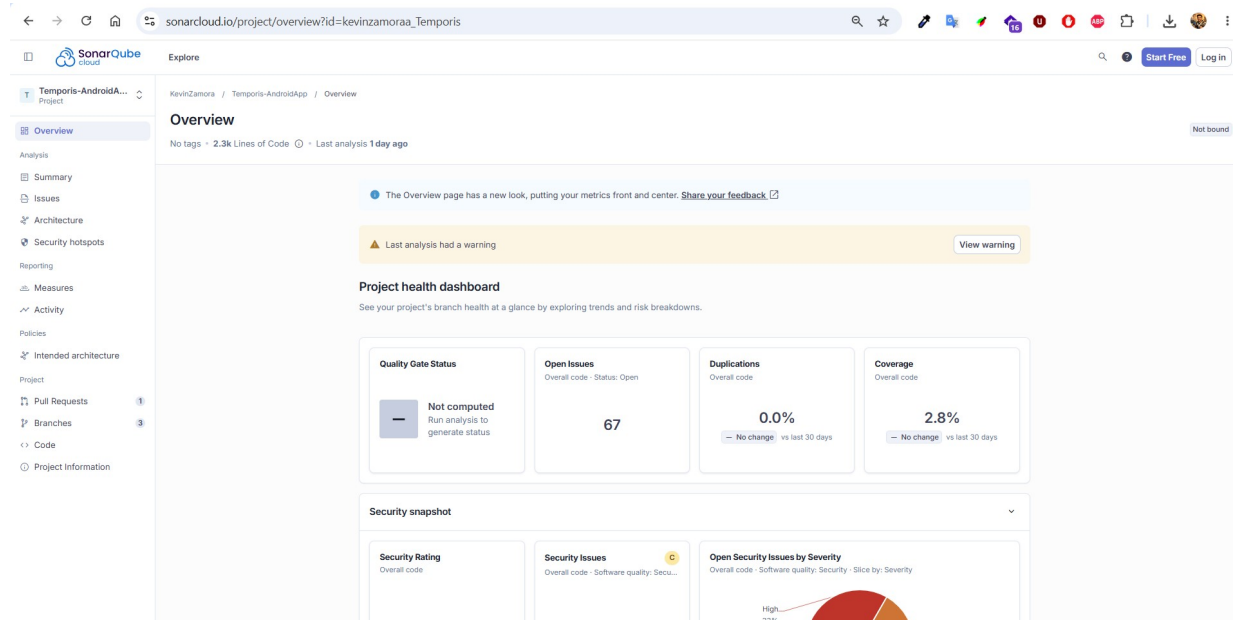
En el siguiente apartado se adjuntan algunas capturas relacionadas con la implementación de ciertos casos de prueba (o *tests*) que hemos empezado a desarrollar, como parte de la validación inicial de la funcionalidad de nuestra aplicación Temporis:

- **Fase 1: Verificación Estática Automática (SonarQube):**

- Importación de datos desde *Github Actions* hacia *Sonar Qube*:



- Panel de control de Sonar Qube:



3. Gestión de proyecto – Planificación

3.1. Planificación Proyecto – Scrum

Para el desarrollo de **Temporis**, se ha adoptado el marco de trabajo ágil **Scrum**, adaptando estratégicamente sus eventos, ceremonias y artefactos a un entorno de desarrollo individual (**Solo-Scrum**). El ciclo de vida del proyecto se divide en *Sprints* fijos de dos semanas de duración. Esta ventana de tiempo garantiza un ritmo de inspección y adaptación constante, permitiendo desplegar incrementos de software potencialmente entregables tras cada ciclo. La flexibilidad de este enfoque facilita la integración inmediata de las pautas de accesibilidad y las correcciones de diseño identificadas durante las fases de prueba continua.

3.1.1. Product Backlog (Pila del Producto)

El *Product Backlog* se compone de historias de usuario y requerimientos técnicos desglosados y ponderados según su valor para el Mínimo Producto Viable (MVP) y su impacto en la inclusión digital. La priorización sigue una taxonomía estricta de dependencias técnicas:

- **Prioridad Alta (Infraestructura y Core CRUD):** Aprovisionamiento del entorno de desarrollo, vinculación de SDKs esenciales de Firebase, y desarrollo del motor asíncrono de temporizadores.
- **Prioridad Media (Interfaz y Accesibilidad):** Maquetación de *layouts* (diseños) con *Material Design 3*, soporte multimodal (modos claro/oscuro) e integración de validación biométrica nativa.
- **Prioridad Baja (Analítica e Información):** Módulo de estadísticas locales, persistencia de históricos globales y tablón de novedades/recursos/publicaciones (*Posts*).

3.1.2. Desarrollo de los Sprints de Ejecución

Sprint 1: Infraestructura Base, Autenticación y Arquitectura

- **Objetivo del Sprint:** Establecer y corregir los cimientos técnicos de la aplicación y asegurar el flujo de acceso de usuarios.
- **Historias de Usuario y Tareas Desarrolladas:**
 - Configuración del proyecto en la consola de *Firebase* e integración del archivo `google-services.json` en el módulo de la *app* Android.
 - Implementación de la arquitectura limpia (*Clean Architecture*) segmentada por capas (*data, domain, ui*).
 - Desarrollo de la interfaz de *login* tradicional (*Email/Contraseña*) mediante *Firebase Auth*.
 - Integración del SDK de *Google Sign-In* para la autenticación social.
 - Implementación de la validación biométrica nativa haciendo uso del *framework* de seguridad *BiometricPrompt*.

Sprint 2: Motor Core de Temporizadores e Interfaz de Usuario Accesible

- **Objetivo del Sprint:** Desarrollar el núcleo operativo de la aplicación y unificar la experiencia visual.
- **Historias de Usuario y Tareas Desarrolladas:**
 - Programación del motor de temporizadores independientes mediante Corrutinas de *Kotlin* (*Flow* y *delay*) para evitar el bloqueo del hilo principal de la UI.
 - Diseño y desarrollo de las vistas funcionales de la aplicación integrando componentes *Material Design 3* (*Cards*, *FAB*, *Bottom Navigation Bar*).
 - Desarrollo de las operaciones CRUD para la sub-colección jerárquica de temporizadores en *Firebase Cloud Firestore*.
 - Implementación de hojas de estilo duales para soportar de forma nativa el Modo Oscuro y el Modo Claro en función de las preferencias del sistema.

Sprint 3: Aseguramiento de la Calidad, Cobertura y Documentación Técnica

- **Objetivo del Sprint:** Validar la estabilidad del software, auditar la deuda técnica y confeccionar la documentación final de entrega.
- **Historias de Usuario y Tareas Desarrolladas:**
 - Construcción de la batería de pruebas unitarias locales para las capas de datos, repositorios y lógica de control de los *ViewModels*.
 - Configuración y ejecución de análisis estáticos de código mediante la integración del *pipeline* con *SonarCloud*.
 - Generación de reportes de cobertura automatizados empleando el *plugin* de *JaCoCo*.
 - Auditoría técnica de accesibilidad visual y compatibilidad con el lector de pantallas *TalkBack* de Android.
 - Redacción integral de la memoria final y del manual técnico del proyecto.

3.1.3. Tablero Kanban (*GitHub Projects*)

Para el seguimiento visual del flujo de trabajo, se ha utilizado un tablero *Kanban* administrado a través de ***GitHub Projects***. Las tareas han transitado de forma transparente por las columnas estandarizadas: *New Issues*, *Ice Box*, *Product Backlog*, *Sprint Backlog*, *In Progress*, *In Review* y *Done*. Esta herramienta ha permitido monitorizar el progreso en tiempo real y detectar cuellos de botella de manera temprana durante las fases críticas de diseño y desarrollo.

4. Gestión del proyecto - Seguimiento

4.1. Seguimiento Proyecto - Scrum

La evaluación continua del desarrollo se ha monitorizado mediante indicadores clave de rendimiento como la velocidad de entrega, la tasa de errores detectados y el cumplimiento estricto de las directrices WCAG de accesibilidad.

4.1.1. Registro y Trazabilidad de Incidencias Técnicas

A continuación, se tabula el historial de incidencias técnicas registradas, analizadas y resueltas durante el ciclo de vida de desarrollo del proyecto:

ID Incidencia	Sprint / Fase	Descripción del Problema Técnico	Solución Aplicada / Mitigación
INC-01	<i>Sprint 1 (Auth)</i>	Error de configuración inicial en los servicios de <i>Firebase</i> y denegación de conexión con el proyecto Android.	Sincronización correcta de las huellas SHA-1 y SHA-256 en la consola de <i>Firebase</i> y actualización del archivo <i>Gradle</i> .
INC-02	<i>Sprint 1 (Logic)</i>	Problemas de compilación en el entorno por directivas de <i>import</i> faltantes tras la migración de entidades a <i>Kotlin</i> .	Refactorización de las clases de datos y corrección de las referencias de dependencias en el IDE.
INC-03	<i>Sprint 2 (Auth)</i>	Error de API código "10" durante los intentos de autenticación cruzada con <i>Google Account</i> .	Configuración explícita del ID de cliente web <i>OAuth</i> dentro de las variables de inicialización del flujo de <i>Firebase</i> .
INC-04	<i>Sprint 2 (UI)</i>	Desajuste visual en la disposición del título de la vista del Perfil de Usuario (<i>User Profile View</i>).	Modificación de las restricciones del <i>Layout</i> para asegurar una visualización simétrica e independiente de la densidad de pantalla.
INC-05	<i>Sprint 2 (Core)</i>	Error crítico en la función de borrado automático de las credenciales locales al invocar el cierre de sesión.	Reajuste en la lógica de limpieza de persistencia local y desconexión controlada del nodo de autenticación remota.
INC-06	<i>Sprint 2 (Core)</i>	Pérdida de sincronización de datos y duplicidad de registros en la base de datos documental <i>Firestore</i> .	Modificación de las funciones de inserción pasando de métodos <i>.add()</i> automáticos a asignación explícita mediante <i>.set()</i> .
INC-07	<i>Sprint 2 (UI)</i>	Desbordamiento de texto (<i>Overflow</i>) en las tarjetas de temporizadores al introducir títulos extensos.	Limitación por software en la longitud del campo (máximo 50 caracteres) e inclusión de elipsis visuales en la interfaz.
INC-08	<i>Sprint 2 (UI)</i>	Contraste insuficiente en la barra superior de la aplicación durante la ejecución bajo el Modo Claro.	Reajuste de los <i>tokens</i> de color base de <i>Material Design 3</i> , garantizando el cumplimiento de los ratios mínimos de la WCAG.
INC-09	<i>Sprint 2 (UI)</i>	Inconsistencia visual y pérdida de estilos en formularios al activar el Modo de Alto Contraste del sistema operativo.	Centralización de estilos en el archivo de temas base de la aplicación, desacoplando los colores de los <i>inputs</i> de las variables volátiles.

4.1.2. Procedimiento de Gestión de Cambios

Ante la detección de desviaciones temporales (provocadas principalmente por la limitada disponibilidad de tiempo operativo o por las iteraciones añadidas para asegurar la accesibilidad), se aplicó de forma estricta el siguiente protocolo de gestión de cambios:

1. **Identificación:** Registro detallado del cambio o corrección requerida en el *Backlog* de *GitHub Issues*.
2. **Análisis de Impacto:** Evaluación técnica sobre el árbol de dependencias arquitectónicas y su repercusión sobre las fechas comprometidas en el Diagrama de *Gantt*.
3. **Repriorización Ágil:** Desplazamiento controlado de tareas estéticas o no esenciales (como animaciones complejas de la interfaz) hacia el *Ice Box* o futuros ciclos de desarrollo, liberando el ancho de banda necesario para solventar las correcciones de accesibilidad o estabilidad estructural sin comprometer la entrega final.

4.1.3. Retrospectiva Final del Proyecto

Al concluir el ciclo ágil, el proceso de retrospectiva arrojó las siguientes conclusiones estratégicas:

- La centralización de los estilos visuales en un archivo de temas unificado ahorra tiempos de desarrollo y previene de forma automatizada los errores de contraste cromático.
- La integración temprana con el ecosistema de *Firebase* facilitó la detección de problemas de latencia en la sincronización desde el primer incremento de *software*.
- La decisión metodológica de reservar de forma exclusiva el *Sprint 3* para tareas de aseguramiento de calidad, refactorización y pruebas automatizadas ha permitido dotar a la versión final de **Temporis** de unos estándares óptimos de mantenibilidad, escalabilidad y robustez técnica industrial.

5. Bibliografía y Webgrafía

5.1. Documentación Oficial y Frameworks de Desarrollo

- **Google Android Developer Documentation.** Guías oficiales de arquitectura de software, *Jetpack Components*, ciclo de vida de componentes (*Activity*, *Fragment*) y persistencia local de datos. Disponible en: <https://developer.android.com>
- **Kotlin Programming Language.** Documentación oficial del lenguaje, gestión de la asincronía mediante corrutinas y flujos asíncronos (*Flow*). *JetBrains*. Disponible en: <https://kotlinlang.org>
- **Google Firebase Architecture.** Documentación técnica sobre la integración de servicios en la nube: especificaciones de autenticación de usuarios mediante *Firebase Auth* y diseño de bases de datos no relacionales documentales en *Cloud Firestore*. Disponible en: <https://firebase.google.com/docs>

5.2. Estándares de Diseño y Accesibilidad Universal

- **Material Design 3 (M3).** Directrices de diseño de interfaces de usuario de Google, especificaciones sobre sistemas de color dinámicos, adaptabilidad y jerarquías visuales orientadas a sistemas Android modernos. Disponible en: <https://m3.material.io>
- **World Wide Web Consortium (W3C).** *Web Content Accessibility Guidelines (WCAG) 2.2*. Estándares e indicadores de accesibilidad internacional aplicados al software móvil (relaciones de contraste lumínico, etiquetado para lectores de pantalla y escalabilidad tipográfica). Disponible en: <https://www.w3.org/WAI/standards-guidelines/wcag/>

5.3. Calidad de Software, Testing y Herramientas de Entorno

- **Robolectric Testing Framework.** Guías de integración y configuración del entorno de pruebas unitarias locales sobre la JVM emulando el SDK de *Android*. Disponible en: <https://robolectric.org>
- **MockK Framework.** Documentación técnica para la implementación de librerías de simulación (*mocking*) enfocadas en código nativo de *Kotlin*. Disponible en: <https://mockk.io>
- **SonarSource & SonarQube.** Métricas de calidad industrial, análisis estático de código para la detección de vulnerabilidades, *code smells* y cálculo de la deuda técnica. Disponible en: <https://www.sonarsource.com>
- **JaCoCo (Java Code Coverage).** Herramientas de automatización de informes de cobertura de líneas de código y de toma de decisiones en bloques condicionales sobre compilaciones *Gradle*. Disponible en: <https://www.eclemma.org/jacoco/>